

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO

BÀI TẬP LỚN CÁC HỆ THỐNG PHÂN TÁN

**CHỦ ĐỀ 3: HỆ THỐNG CHAT NGANG HÀNG (P2P)
HYBRID PHÂN TÁN**

Nhóm thực hiện: **13**

THÀNH VIÊN THỰC HIỆN

1. Nguyễn Duy Hà - B22DCCN256
2. Nguyễn Mạnh Tuấn - B22DCCN760
3. Phạm Quang Dũng - B22DCCN136

Giảng viên bộ môn: **Kim Ngọc Bách**

Hà Nội – 2/2026

LỜI CẢM ƠN

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến các thầy cô giáo tại Học viện Công nghệ Bưu chính Viễn thông, đặc biệt là các thầy cô trong Khoa Công nghệ thông tin, vì sự tận tâm giảng dạy và truyền đạt cho chúng em những kiến thức, kinh nghiệm quý báu trong suốt quá trình học tập.

Đặc biệt, chúng em xin gửi lời cảm ơn chân thành nhất đến Thầy Kim Ngọc Bách – giảng viên bộ môn Hệ thống phân tán. Thầy đã nhiệt tình hướng dẫn, hỗ trợ và chỉ bảo chúng em trong suốt quá trình thực hiện đề án môn học. Trong thời gian học tập và trao đổi cùng Thầy, chúng em không chỉ thu nhận được nhiều kiến thức chuyên môn bổ ích về mô hình mạng ngang hàng P2P, cơ chế truyền thông và tính chịu lỗi phân tán, mà còn học hỏi được tinh thần làm việc nghiêm túc cùng thái độ nghiên cứu khoa học hiệu quả. Những điều kiện này chắc chắn sẽ là hành trang vô cùng quý giá cho chúng em trong tương lai.

Tuy đã có nhiều cố gắng, nhưng do thời gian và kiến thức còn hạn chế nên bài báo cáo chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý, chỉ bảo thêm của Thầy để hệ thống được hoàn thiện hơn.

Nhóm thực hiện đề án xin chân thành cảm ơn thầy!

MỤC LỤC

LỜI CẢM ON.....	2
DANH MỤC HÌNH VẼ.....	4
DANH MỤC BẢNG BIỂU.....	5
DANH MỤC TỪ VIẾT TẮT.....	6
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI.....	8
1.1 Thực trạng và vấn đề cần giải quyết.....	8
1.2. Lý do chọn đề tài & tính cấp thiết.....	9
1.3. Mục tiêu và phạm vi đề tài.....	10
1.4. Đối tượng và người dùng hệ thống.....	12
1.5. Bố cục các chương.....	13
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT & CÔNG NGHỆ.....	15
2.1. Mô hình hệ thống P2P Hybrid.....	15
2.2. Nền tảng lý thuyết các thuật toán áp dụng.....	17
2.3. Công nghệ và công cụ phát triển.....	21
CHƯƠNG 3: PHÂN TÍCH HỆ THỐNG.....	23
3.1. Yêu cầu chức năng và phi chức năng.....	23
3.2. Sơ đồ Use Case.....	27
CHƯƠNG 4: THIẾT KẾ HỆ THỐNG & CẤU TRÚC DỮ LIỆU.....	33
4.1. Kiến trúc hệ thống phân lớp.....	34
4.2. Thiết kế Giao thức trao đổi thông điệp (Message Protocol).....	36
4.3. Thiết kế Cơ chế Vận hành và Định tuyến.....	40
4.4. Thiết kế Cơ chế Chịu lỗi và Đảm bảo độ tin cậy (Fault Tolerance & Reliability)	43
4.5. Tổng kết luồng hoạt động vòng đời hệ thống (Lifecycle).....	46
CHƯƠNG 5: CÀI ĐẶT - KIỂM THỬ.....	49
5.1. Triển khai và Cài đặt.....	49
5.2. Kịch bản và Kết quả Kiểm thử (Testing & Evaluation).....	58
CHƯƠNG 6: KẾT LUẬN & HƯỚNG PHÁT TRIỂN.....	63
6.1. Tổng kết kết quả đạt được.....	63
6.2. Phương hướng phát triển.....	65
TÀI LIỆU THAM KHẢO.....	67
PHỤ LỤC.....	69

DANH MỤC HÌNH VẼ

Hình 3.1. Sơ đồ Use Case tổng quát của hệ thống Chat P2P Hybrid

Hình 4.1. Sơ đồ kiến trúc tổng thể của hệ thống Chat P2P Hybrid

Hình 4.2. Sơ đồ lớp - Kiến trúc chi tiết hệ thống P2P

Hình 4.3. Sơ đồ tuần tự khám phá peer (LAN và WAN)

Hình 4.4. Sơ đồ Tuần tự: Tiến trình trao đổi khóa ECDH

Hình 4.5. Sơ đồ Tuần tự Tổng quát (Master Sequence Diagram)

Hình 5.1. Giao diện người dùng ứng dụng P2P Chat

Hình 5.2. Thử nghiệm truyền thông điệp mã hóa trực tiếp (Direct Chat) giữa hai Node

Hình 5.3. Giao diện thử nghiệm Broadcast message toàn mạng

Hình 5.4. Kịch bản truyền dữ liệu đa phương tiện (Hình ảnh) qua kênh mã hóa E2EE

Hình 5.5. Hộp thoại trích xuất cấu trúc dữ liệu mật mã thực tế của gói tin

Hình 5.6. Giao diện Tạo phòng

Hình 5.7. Giao diện Tạo phòng thành công (mã room key - d26#q5ky)

Hình 5.8. Giao diện phòng D26 sau khi tạo thành công

Hình 5.9. Giao diện Gia nhập phòng (Join Room)

Hình 5.10. Giao diện sau khi nhập keyroom và vào thành công phòng D26

Hình 5.11. Chụp kết quả log Terminal - BÁO CÁO CHURN SIMULATION

DANH MỤC BẢNG BIỂU

Bảng 1.1 Thực trạng và vấn đề cần giải quyết.

Bảng 1.2. Đối tượng và người dùng hệ thống.

Bảng 2.1. Công nghệ và công cụ phát triển.

Bảng 3.1. Bảng yêu cầu phi chức năng.

Bảng 4.1. Đặc tả các loại thông điệp.

Bảng 5.1. Thống kê kết quả thực nghiệm tự động (Automated Test Metrics).

DANH MỤC TỪ VIẾT TẮT

P2P	Peer-to-Peer (ngang hàng)
Hybrid P2P	Mạng ngang hàng lai
E2EE	End-to-End Encryption (mã hóa đầu cuối)
ECDH	Elliptic Curve Diffie-Hellman (trao đổi khóa đường cong elliptic)
AES	Advanced Encryption Standard (chuẩn mã hóa tiên tiến)
GCM	Galois/Counter Mode (chế độ GCM)
SHA-256	Secure Hash Algorithm 256-bit
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
LAN	Local Area Network (mạng nội bộ)
WAN	Wide Area Network (mạng diện rộng)
GUI	Graphical User Interface (giao diện đồ họa)
IPC	Inter-Process Communication (giao tiếp liên tiến trình)
ACK	Acknowledgment (xác nhận)
TTL	Time-To-Live (thời gian sống)
KDF	Key Derivation Function (hàm dẫn xuất khóa)
AEAD	Authenticated Encryption with Associated Data

MAC	Message Authentication Code
DTN	Delay-Tolerant Networking (mạng chịu trễ)
ARQ	Automatic Repeat reQuest (tự động gửi lại)
PKI	Public Key Infrastructure (hạ tầng khóa công khai)
DID	Decentralized Identifier (định danh phi tập trung)
STUN	Session Traversal Utilities for NAT
TURN	Traversal Using Relays around NAT
WebRTC	Web Real-Time Communication
VPS	Virtual Private Server (máy chủ riêng ảo)
RAM	Random Access Memory (bộ nhớ truy cập ngẫu nhiên)
DOM	Document Object Model
JSON	JavaScript Object Notation
JS	JavaScript
Node.js	Môi trường thực thi JavaScript phía máy chủ
Electron	Framework xây dựng ứng dụng desktop đa nền tảng
NIST	National Institute of Standards and Technology (Hoa Kỳ)
IETF	Internet Engineering Task Force
RFC	Request for Comments

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Thực trạng và vấn đề cần giải quyết

Trong kỷ nguyên số, các ứng dụng nhắn tin thời gian thực đã trở thành phương tiện liên lạc chủ đạo. Tuy nhiên, phần lớn các nền tảng phổ biến hiện nay (Zalo, Messenger, Telegram, WhatsApp...) đều vận hành theo mô hình Client-Server tập trung. Mô hình này bộc lộ nhiều hạn chế mang tính hệ thống, đặc biệt khi áp dụng vào các ngữ cảnh yêu cầu cao về bảo mật và khả năng tự trị.

Vấn đề	Mô tả chi tiết	Hậu quả
Single Point of Failure	Máy chủ trung tâm là điểm hội tụ duy nhất của toàn bộ lưu lượng điều khiển và dữ liệu	Hệ thống tê liệt hoàn toàn khi máy chủ gặp sự cố kỹ thuật, bị tấn công DDoS hoặc bảo trì định kỳ
Quyền riêng tư dữ liệu	Toàn bộ nội dung hội thoại được lưu trữ và kiểm soát bởi đơn vị vận hành máy chủ (bên thứ ba)	Nguy cơ rò rỉ dữ liệu hàng loạt, bị khai thác trái phép cho mục đích thương mại hoặc giám sát, bất chấp các tuyên bố về mã hóa "đầu cuối"
Chi phí hạ tầng	Máy chủ trung tâm phải định tuyến và xử lý toàn bộ luồng tin nhắn giữa những người dùng.	Chi phí băng thông, tài nguyên tính toán và vận hành tăng theo cấp số nhân khi lượng người dùng mở rộng.
Phụ thuộc Internet	Mọi tương tác đều yêu cầu đường truyền ổn định đến máy chủ trung tâm đặt trên Internet công cộng.	Hệ thống không thể hoạt động trong các môi trường mạng nội bộ bị cô lập (intranet), mạng có kết nối Internet hạn chế hoặc các tình huống khẩn cấp mất liên lạc diện rộng.

Bảng 1.1. Thực trạng và vấn đề cần giải quyết

Rõ ràng, các mô hình tập trung truyền thống không thể đáp ứng trọn vẹn các yêu cầu về tính riêng tư tuyệt đối, khả năng chịu lỗi cao và tính tự trị trong môi trường mạng cục bộ. Điều này thúc đẩy nhu cầu nghiên cứu và phát triển một kiến trúc truyền thông phi tập trung, nơi các bên tham gia có thể trao đổi dữ liệu trực tiếp mà không cần phó thác hoàn toàn vào một thực thể thứ ba.

1.2. Lý do chọn đề tài & tính cấp thiết

Kiến trúc Peer-to-Peer (P2P) cung cấp một cách tiếp cận khác biệt: các thiết bị đầu cuối (peer) tự tổ chức thành một mạng phủ (overlay network) và giao tiếp trực tiếp với nhau. Mô hình Hybrid P2P được lựa chọn cho đề án này, trong đó một Bootstrap Server nhẹ chỉ đóng vai trò "danh bạ" khởi tạo, hỗ trợ các peer tìm thấy nhau, còn toàn bộ dữ liệu người dùng (tin nhắn, file) được truyền trực tiếp qua các kênh P2P.

Đề tài "Phát triển hệ thống chat ngang hàng P2P Hybrid phân tán" mang tính cấp thiết cao, xuất phát từ các luận cứ sau:

- Bảo mật và quyền riêng tư là ưu tiên số một: Hệ thống tích hợp mã hóa đầu-cuối (End-to-End Encryption – E2EE) sử dụng cơ chế trao đổi khóa ECDH và thuật toán AES-256-GCM. Điều này đảm bảo rằng ngay cả Bootstrap Server cũng không thể đọc được nội dung hội thoại – một yêu cầu sống còn với các tổ chức chính phủ, ngân hàng, quân đội và doanh nghiệp coi trọng bí mật thông tin.
- Khả năng chịu lỗi và hoạt động độc lập: Nhờ kiến trúc P2P, các peer đã biết nhau có thể tiếp tục liên lạc ngay cả khi Bootstrap Server gặp sự cố. Cơ chế Store-and-Forward cho phép lưu tạm tin nhắn trên các peer trung gian và tự động chuyển tiếp khi người nhận trực tuyến trở lại, giải quyết bài toán nhắn tin bất đồng bộ trong môi trường phân tán không ổn định.
- Ứng dụng trong môi trường đặc thù: Hệ thống hoạt động hiệu quả trên mạng LAN mà không cần kết nối Internet, rất phù hợp cho các mạng nội bộ bị cô lập, hệ thống liên lạc tác chiến, hay mạng cảm biến trong vùng sâu vùng xa – nơi hạ tầng mạng công cộng không khả dụng.
- Tính thực tiễn trong đào tạo và nghiên cứu: Đề án là một minh chứng hoàn chỉnh cho việc áp dụng các nguyên lý của Hệ thống phân tán (distributed systems) vào bài toán thực tế: giao tiếp liên tiến trình qua TCP/UDP, đồng bộ trạng thái phi tập trung, phát hiện lỗi bằng heartbeat, và xây dựng giao thức tầng ứng dụng.

Việc xây dựng thành công một hệ thống như vậy không chỉ khẳng định năng lực làm chủ công nghệ lõi của sinh viên mà còn tạo ra một nền tảng mã nguồn mở có giá trị cho cộng đồng nghiên cứu về mạng ngang hàng bảo mật.

1.3. Mục tiêu và phạm vi đề tài

Căn cứ vào yêu cầu đề án và kết quả đã triển khai, các mục tiêu của hệ thống được phân định rõ ràng thành hai nhóm: yêu cầu cơ bản và yêu cầu nâng cao.

1.3.1. Mục tiêu cơ bản

- Tham gia mạng P2P: Peer đăng ký với Bootstrap Server qua TCP, nhận PEER_LIST và tự động thiết lập kết nối P2P đến các peer khác.
- Chat trực tiếp giữa các peer: Gửi/nhận tin nhắn văn bản trực tiếp qua kênh TCP P2P, có mã hóa E2EE và cơ chế ACK đảm bảo độ tin cậy.
- Chat nhóm: Hỗ trợ tạo/tham gia phòng chat, gửi tin nhắn đến tất cả thành viên trong nhóm qua kỹ thuật Multi-Unicast.
- Peer discovery: Tự động phát hiện peer qua UDP Broadcast (trong LAN) và đồng bộ danh sách từ Bootstrap Server (trong WAN)
- Trạng thái online/offline: Duy trì và cập nhật trạng thái peer theo thời gian thực nhờ heartbeat (PING/PONG) và cơ chế phát hiện stale peer.
- Truyền tin đáng tin cậy: Đảm bảo gửi tin thành công qua sequence number, deduplication, retry (3 lần) và cơ chế Store-and-Forward khi peer đích offline.

1.3.2. Mục tiêu nâng cao

- Broadcast message trong toàn mạng: Cho phép gửi một thông điệp đến tất cả peer đang online (GlobalChat) với cơ chế kiểm soát vòng lặp bằng Message ID và TTL.
- Store-and-forward messaging khi peer offline: Tin nhắn gửi khi peer đích offline được lưu tạm trên các peer trung gian (relayQueue) và tự động chuyển tiếp khi đích online trở lại.
- Mã hóa tin nhắn (encryption): Sử dụng ECDH (prime256v1) để trao đổi khóa an toàn và AES-256-GCM để mã hóa/xác thực toàn vẹn từng gói tin.
- Giao diện GUI: Cung cấp ứng dụng desktop cross-platform xây dựng bằng Electron, hỗ trợ đầy đủ các thao tác chat, tạo phòng, gửi file, hiển thị trạng thái bảo mật.

- File transfer giữa các peer: Cho phép gửi file ($\leq 5\text{MB}$) dưới dạng Base64 tích hợp trong luồng chat, kế thừa toàn bộ cơ chế mã hóa E2EE của Direct Chat.
- Mô phỏng churn (peer tham gia và rời mạng liên tục): Cung cấp script churn-test.js để đánh giá độ ổn định của hệ thống khi các peer liên tục tham gia và rời mạng.

1.3.3. Phạm vi và giới hạn

Hệ thống được phát triển như một sản phẩm hoàn thiện ở mức đồ án môn học, tập trung thể hiện các nguyên lý cốt lõi của hệ phân tán. Các giới hạn kỹ thuật hiện tại – được ghi nhận một cách minh bạch – bao gồm:

- Hệ thống vẫn tồn tại một điểm yếu tập trung (single point of failure) ở Bootstrap Server trong pha khởi tạo mạng WAN. Điều này được khắc phục bằng cơ chế fallback qua UDP Broadcast trong LAN và cơ chế học danh tính ngầm qua TCP, giúp các peer đã biết nhau vẫn hoạt động bình thường khi server sập. Tuy nhiên, peer mới muốn tham gia mạng từ xa vẫn cần server. Giải pháp dài hạn là triển khai nhiều Bootstrap Server kết hợp cơ chế failover
- Truyền file: Sử dụng mã hóa Base64 (tăng kích thước $\sim 33\%$) và giới hạn ở mức 5MB do chưa cài đặt cơ chế chia nhỏ (chunking) và truyền luồng (streaming) cho file lớn.
- NAT Traversal: Hiện chưa tích hợp STUN/TURN; các kết nối xuyên NAT được thực hiện gián tiếp qua relay của Bootstrap Server. Đây là hướng phát triển tương lai với WebRTC.
- Lưu trữ bền vững (Persistence): Hàng đợi Store-and-Forward cùng trạng thái peer hiện được lưu trong bộ nhớ RAM. Khi một peer tắt đột ngột, các tin nhắn đang giữ hộ có thể bị mất.
- Xác thực danh tính: Hệ thống chưa tích hợp cơ chế xác thực mạnh (PKI, DID). Định danh peer hiện dựa trên ID tự sinh, chưa chống giả mạo tuyệt đối.

- Giao diện người dùng: GUI được xây dựng bằng Vanilla JavaScript để phục vụ mục tiêu kiểm thử và demo, chưa được tối ưu về UX/UI cho người dùng phổ thông.

Những giới hạn trên không ảnh hưởng đến việc đáp ứng toàn bộ yêu cầu chức năng của đề án mà chính là các định hướng mở rộng rõ ràng cho các phiên bản tiếp theo.

1.4. Đối tượng và người dùng hệ thống

Hệ thống được thiết kế hướng tới các nhóm đối tượng đa dạng, mỗi nhóm hưởng lợi từ những khía cạnh khác nhau của kiến trúc Hybrid P2P bảo mật:

Đối tượng	Nhu cầu	Lợi ích từ hệ thống
Người dùng cá nhân	Quan tâm đến quyền riêng tư, không muốn dữ liệu hội thoại bị lưu trữ và phân tích bởi các tập đoàn công nghệ.	Mã hóa E2EE đảm bảo chỉ người gửi và người nhận đọc được nội dung. Không có server trung tâm lưu trữ plaintext.
Doanh nghiệp nội bộ	Cần một kênh liên lạc bảo mật, hoạt động độc lập, không phụ thuộc vào hạ tầng Internet công cộng (ví dụ: mạng nội bộ trong tòa nhà).	Hoạt động hoàn toàn trong LAN, không cần đường truyền Internet. Dữ liệu không bao giờ rời khỏi hệ thống mạng của tổ chức.
Tổ chức đặc thù (quân đội, cứu hộ, vùng sâu vùng xa)	Yêu cầu liên lạc trong môi trường mạng bị cô lập, cơ động, địa hình phức tạp, hoặc sau thảm họa.	Kiến trúc P2P mesh tự tổ chức, chịu lỗi cao, không phụ thuộc vào bất kỳ điểm trung tâm nào sau khi khởi tạo. Store-and-Forward đảm bảo thông điệp đến được đích.
Sinh viên, Nhà nghiên cứu	Cần một nền tảng thực nghiệm để học tập, kiểm chứng các nguyên lý hệ thống phân tán, mạng ngang hàng, mật mã ứng dụng.	Mã nguồn mở hoàn toàn, kiến trúc module hóa rõ ràng, dễ dàng mở rộng, tích hợp và kiểm thử

Bảng 1.2. Đối tượng và người dùng hệ thống

1.5. Bố cục các chương

Báo cáo đồ án được tổ chức thành 6 chương:

- Chương 1 – Giới thiệu đề tài: Đặt vấn đề, khẳng định tính cấp thiết, xác định mục tiêu và phạm vi đã hoàn thành của đồ án.
- Chương 2 – Cơ sở lý thuyết và công nghệ: Cung cấp nền tảng lý thuyết học thuật về mô hình Hybrid P2P, định nghĩa cơ bản về các thuật toán áp dụng (Controlled Flooding/Broadcast, ECDH, AES-256-GCM, Store-and-Forward Routing, Failure Detection) và tổng quan về các công nghệ nền tảng được sử dụng (Node.js, Electron, TCP/UDP socket).
- Chương 3 – Phân tích hệ thống: Phân tích chi tiết các yêu cầu chức năng và phi chức năng. Tập trung xây dựng mô hình hóa yêu cầu thông qua Sơ đồ Use Case, qua đó làm rõ các tác nhân tham gia và các luồng chức năng người dùng cần tương tác với hệ thống.
- Chương 4 – Thiết kế hệ thống và cấu trúc dữ liệu: Đây là chương trọng tâm, đặc tả chi tiết giải pháp kỹ thuật:
 - + Kiến trúc hệ thống: Mô tả kiến trúc phân lớp (Core, Security, Network, Chat Layer). Sử dụng Sơ đồ Lớp (Class Diagram) để làm rõ mối quan hệ đồng lập và phụ thuộc giữa các đối tượng cốt lõi như Peer, TCPHandler, UDPHandler, BootstrapClient.
 - + Giao thức trao đổi thông điệp: Đặc tả hoàn chỉnh giao thức độc quyền JSON-over-TCP (newline-delimited) với bảng mã loại thông điệp (REGISTER, DIRECT_CHAT, STORE_AND_FORWARD...), cấu trúc payload, và cách thức tích hợp mã hóa E2EE vào gói tin.
 - + Cơ chế vận hành và Định tuyến: Sử dụng Sơ đồ Tuần tự (Sequence Diagram) để phân tích cơ chế khám phá peer (UDP Broadcast cho LAN, Bootstrap Server Sync cho WAN, Học peer ẩn qua TCP handshake).
 - + Cơ chế Đảm bảo độ tin cậy và Chịu lỗi: Thiết kế cấu trúc luồng xử lý lỗi bao gồm: Hàng đợi MessageQueue (với sequence number, ACK), cơ chế phát hiện lỗi (heartbeat PING/PONG) và giải pháp định tuyến phục hồi Store-and-Forward.

- Chương 5 – Triển khai cài đặt và Kiểm thử: Trình bày quá trình hiện thực hóa bản thiết kế và đánh giá hệ thống:
 - + Triển khai cài đặt: Mô tả cấu trúc mã nguồn, trích dẫn các đoạn code (snippets) xử lý nghiệp vụ cốt lõi.
 - + Thử nghiệm hệ thống: Kịch bản kiểm thử toàn diện: đăng ký & khám phá, chat E2EE, Store-and-Forward , mô phỏng churn, và truyền file và các chức năng liên quan. Các kết quả thực nghiệm (độ trễ, tỉ lệ gửi thành công) được thống kê và phân tích nhằm chứng minh hệ thống đáp ứng các yêu cầu đặt ra.
- Chương 6 – Kết luận và hướng phát triển: Tổng kết các kết quả kỹ thuật đã đạt được và đề xuất lộ trình phát triển nâng cấp cho hệ thống.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT & CÔNG NGHỆ

2.1. Mô hình hệ thống P2P Hybrid

2.1.1. Phân loại kiến trúc P2P

Hệ thống ngang hàng (Peer-to-Peer) là mô hình mạng phân tán trong đó các nút (peer) chia sẻ tài nguyên và giao tiếp trực tiếp với nhau mà không cần thông qua một máy chủ trung tâm làm trung gian cho dữ liệu. Dựa trên cách thức tổ chức và khám phá peer, kiến trúc P2P được chia thành ba loại chính:

- P2P thuần túy (Pure P2P): Không có bất kỳ thành phần tập trung nào. Mọi peer đều bình đẳng, tự tổ chức thành mạng phủ (overlay) và sử dụng các giao thức flooding hoặc Distributed Hash Table (DHT) để định tuyến.
 - + Ví dụ: Gnutella (flooding), Chord (DHT).
 - + Ưu điểm: Không có điểm yếu đơn lẻ (single point of failure).
 - + Nhược điểm: Khởi tạo mạng phức tạp, hiệu năng tìm kiếm thấp nếu flooding không kiểm soát.
- P2P lai (Hybrid P2P): Sử dụng một hoặc một số máy chủ trung tâm nhẹ (thường gọi là Bootstrap Server, Tracker) để hỗ trợ quá trình khám phá peer và điều phối kết nối ban đầu. Dữ liệu thực tế (tin nhắn, file) vẫn được truyền trực tiếp giữa các peer.
 - + Ví dụ: BitTorrent (Tracker), Skype (trước đây).
 - + Ưu điểm: Kết hợp được tính đơn giản trong quản lý của client-server và tính phân tán của P2P.
 - + Nhược điểm: Vẫn tồn tại một điểm tập trung yếu, nhưng ảnh hưởng được giới hạn ở pha khởi tạo.
- P2P phân cấp (Hierarchical P2P): Các peer được tổ chức thành các nhóm, mỗi nhóm có một siêu nút (super-peer) đảm nhận vai trò điều phối. Các siêu nút lại kết nối với nhau.
 - + Ưu điểm: Cân bằng tải, giảm flooding.
 - + Nhược điểm: Phức tạp trong bầu chọn và duy trì siêu nút.

2.1.2. Mô hình Hybrid P2P của đồ án

Sau khi phân tích ưu và nhược điểm của các kiến trúc mạng hiện có, đồ án quyết định lựa chọn Mô hình Hybrid P2P (Mạng ngang hàng lai) làm nền tảng kiến

trúc cốt lõi cho hệ thống ứng dụng Chat. Quyết định này xuất phát từ nhu cầu giải quyết trọn vẹn bài toán cân bằng giữa hiệu năng định tuyến (Routing Efficiency) và quyền riêng tư dữ liệu (Data Privacy).

Trong cấu trúc Hybrid P2P được áp dụng tại đồ án, hệ thống không loại bỏ hoàn toàn máy chủ trung tâm mà phân tách rõ rệt vai trò của các thành phần tham gia thành hai luồng hoạt động song song: Luồng Điều khiển (Control Plane) và Luồng Dữ liệu (Data Plane).

1. Luồng Điều khiển tập trung (Centralized Control Plane): Thay vì sử dụng thuật toán truyền bá ngập lụt (Flooding) rủi ro và tốn kém băng thông để tìm kiếm nhau như trong mô hình Pure P2P (Mạng ngang hàng thuần túy), đồ án sử dụng một máy chủ trung tâm trọng lượng nhẹ gọi là Bootstrap Server (Tracker/Registry).
 - Vai trò: Máy chủ này đóng vai trò như một "cuốn danh bạ điện thoại". Nhiệm vụ duy nhất của nó là lưu trữ siêu dữ liệu (Metadata) tĩnh như định danh (Peer ID), địa chỉ IP và Cổng mạng (Port) của các Node đang trực tuyến (Online).
 - Đặc tính bảo mật: Máy chủ hoàn toàn không tham gia vào quá trình lưu trữ hay trung chuyển nội dung tin nhắn của người dùng. Điều này loại bỏ hoàn toàn nguy cơ rò rỉ dữ liệu từ máy chủ trung tâm – một điểm yếu cố hữu của mô hình Client-Server truyền thống.
2. Luồng Dữ liệu phân tán (Distributed Data Plane): Sau khi các Node (thực thể người dùng) nhận được thông tin định tuyến từ Bootstrap Server, chúng sẽ bỏ qua máy chủ trung tâm và trực tiếp thiết lập các liên kết điểm-đến-điểm (Point-to-Point) với nhau.
 - Vai trò: Toàn bộ dữ liệu nặng và nhạy cảm bao gồm văn bản trò chuyện (Text), tệp tin đa phương tiện (File/Media) đều được truyền tải trực tiếp qua các kết nối TCP Sockets ngang hàng này.
 - Hiệu năng: Việc trung chuyển trực tiếp giúp giảm thiểu độ trễ (Latency), tối ưu hóa băng thông (Bandwidth) do không bị thất cổ chai tại máy chủ trung tâm, đồng thời tạo ra môi trường lý tưởng để áp dụng thuật toán Mã hóa đầu cuối (E2EE - End-to-End Encryption).

3. **Đặc tính Sinh tồn mạng nội bộ (LAN Resilience):** Một điểm nhấn đặc biệt chứng minh tính ưu việt của kiến trúc Hybrid trong đồ án này là khả năng thoái lui động (Dynamic Fallback) về trạng thái Pure P2P. Trong trường hợp luồng Điều khiển tập trung (Bootstrap Server) gặp sự cố đứt kết nối diện rộng (WAN Failure), các Node cục bộ trong cùng một dải mạng LAN có khả năng tự động kích hoạt cơ chế dò tìm bằng sóng vô tuyến (UDP Broadcast). Hệ thống lúc này hoạt động độc lập hoàn toàn mà không cần kết nối Internet, đảm bảo tính liên tục của dịch vụ thông tin liên lạc.
4. Để tăng cường khả năng chịu lỗi cho Bootstrap Server, hệ thống hỗ trợ cơ chế fallback qua UDP Broadcast trong LAN và cơ chế học danh tính ngầm qua TCP. Khi server không khả dụng, các peer trong cùng mạng LAN vẫn có thể khám phá lẫn nhau; đồng thời, các peer đã từng kết nối vẫn duy trì được kết nối TCP trực tiếp. Phiên bản tiếp theo sẽ triển khai nhiều instance Bootstrap Server kết hợp failover để loại bỏ hoàn toàn SPOF.

Nhờ sự kết hợp khéo léo này, mô hình Hybrid P2P của đồ án vừa khắc phục được nhược điểm định tuyến chậm chạp của Pure P2P, vừa phá bỏ được rào cản tắc nghẽn dữ liệu tập trung của Client-Server, tạo ra một kiến trúc lý tưởng cho các ứng dụng giao tiếp thời gian thực an toàn.

2.2. Nền tảng lý thuyết các thuật toán áp dụng

Hệ thống chat P2P Hybrid bảo mật là sự kết hợp chặt chẽ giữa các thuật toán mật mã học hiện đại và các cơ chế điển hình của hệ thống phân tán. Mỗi thuật toán được lựa chọn không chỉ vì tính an toàn, hiệu năng mà còn vì sự phù hợp với mô hình truyền thông phi tập trung, nơi không có bên thứ ba đáng tin cậy.

2.2.1. Nhóm thuật toán Mật mã học (Bảo mật E2EE)

Để đảm bảo tính riêng tư và toàn vẹn cho mọi nội dung trao đổi ngay cả trên kênh truyền không an toàn, hệ thống triển khai cơ chế mã hóa đầu-cuối (End-to-End Encryption) kết hợp sức mạnh của mật mã bất đối xứng và đối xứng.

a. Thuật toán trao đổi khóa ECDH (Elliptic Curve Diffie-Hellman)

- Nguyên lý hoạt động:

- + ECDH là giao thức thỏa thuận khóa dựa trên đường cong elliptic. Hai peer (A và B) mỗi bên tự sinh một cặp khóa riêng tư – công khai trên cùng một đường cong. Sau khi trao đổi public key cho nhau qua kênh công khai, mỗi bên kết hợp khóa riêng của mình với khóa công khai của đối phương để cùng tạo ra một shared secret giống hệt nhau.
- + Kẻ nghe lén chỉ biết hai public key, không thể tính được shared secret nếu không giải được bài toán logarit rời rạc elliptic.
- Lý do áp dụng:
 - + Hiệu năng và băng thông tối ưu: Với độ dài khóa chỉ 256 bit, các đường cong Elliptic như prime256v1 (NIST P-256) và secp256k1 cung cấp mức độ bảo mật tương đương khóa RSA 3072 bit. Điều này giúp giảm tải đáng kể băng thông truyền tải trên mạng P2P vốn phải đính kèm khóa công khai (Public Key) trong gói tin bắt tay khởi tạo ban đầu.
 - + Không cần kênh an toàn sẵn có: ECDH giải quyết bài toán nan giải “làm sao trao chìa khóa cho bạn mà không bị hacker chặn mất” – là nền tảng cốt lõi của mọi hệ thống E2EE, đảm bảo ngay cả máy chủ trung gian Bootstrap Server cũng không thể biết được khóa phiên của cuộc hội thoại.
 - + Khả năng tương thích linh hoạt và tính tự trị (Dynamic Fallback): Hệ thống ưu tiên chọn prime256v1 – đường cong tiêu chuẩn quốc tế được khuyến nghị bởi NIST và hỗ trợ mặc định 100% trên hầu hết các nền tảng Node.js cũng như Electron Desktop. Đồng thời, hệ thống tích hợp cơ chế tự động đàm phán thông minh: nếu môi trường chạy của thiết bị gặp hạn chế, nó sẽ tự động thoái lui (fallback) sang sử dụng đường cong secp256k1 (đường cong mật mã nổi tiếng được sử dụng trong mạng lưới Bitcoin). Cơ chế này giúp mạng lưới duy trì tính sinh tồn cao và hoạt động thông suốt trên nhiều cấu hình thiết bị khác nhau.

b. Thuật toán mã hóa đối xứng AES-256-GCM và hàm băm SHA-256 (Secure Hash Algorithm)

- Nguyên lý hoạt động:

- + AES (Advanced Encryption Standard) mã hóa dữ liệu theo khối 128 bit với khóa 256 bit. Chế độ GCM (Galois/Counter Mode) kết hợp giữa mã hóa dòng (counter mode) và xác thực (Galois field) để tạo ra một authenticated encryption with associated data (AEAD). Cụ thể, mỗi lần mã hóa, ngoài bản mã còn sinh ra một authentication tag (MAC). Khi giải mã, tag này được kiểm tra để xác nhận dữ liệu không bị sửa đổi và đến từ bên giữ đúng khóa.
- + SHA-256 là hàm băm mật mã một chiều, nhận đầu vào có độ dài bất kỳ và tạo ra một giá trị băm cố định 256 bit (32 byte). Các đặc tính quan trọng: kháng tiền ảnh (không thể tìm ngược đầu vào từ giá trị băm) kháng va chạm (khó tìm hai đầu vào khác nhau cho cùng một băm).
- Lý do áp dụng:
 - + Tính bảo mật: AES-256 được chính phủ Hoa Kỳ phê chuẩn cho tài liệu tối mật (TOP SECRET).
 - + Tính toàn vẹn: Nhờ GCM, bất kỳ sự can thiệp nào vào bản mã trên đường truyền đều bị phát hiện ngay khi giải mã, ngăn chặn tấn công sửa đổi.
 - + Hiệu năng phần cứng: AES được tăng tốc trực tiếp trên CPU hiện đại (AES-NI), giúp mã hóa/giải mã gần như tức thời, không ảnh hưởng trải nghiệm chat thời gian thực.
 - + Shared secret sinh ra từ ECDH là một điểm trên đường cong, chưa có cấu trúc cố định và có thể chứa những điểm yếu về mặt thống kê. Bằng cách băm shared secret này qua SHA-256, ta thu được một khóa AES 256-bit phân bố đều và an toàn về mặt mật mã, sẵn sàng làm việc với AES-256-GCM.

2.2.2. Nhóm thuật toán & Cơ chế Phân tán (Distributed Systems)

Một hệ thống P2P thực thụ đòi hỏi nhiều hơn là mã hóa dữ liệu; nó phải có khả năng tự tổ chức, định tuyến thông minh và phát hiện lỗi trong môi trường không có trung tâm điều phối.

a. Thuật toán Truyền bá có kiểm soát (Controlled Flooding / Broadcast)

- Nguyên lý: Flooding là kỹ thuật định tuyến đơn giản nhất trong mạng P2P: một nút muốn gửi thông điệp đến toàn mạng sẽ chuyển tiếp nó cho

tất cả các nút láng giềng, và cứ thế lan rộng. Tuy nhiên, flooding thuần túy dễ gây ra broadcast storm (bão mạng) và lặp vô hạn.

- Biến thể áp dụng – Controlled Flooding:

1. Deduplication bằng Message ID: Mỗi thông điệp có một id duy nhất. Peer duy trì tập seenMessageIds. Khi nhận được thông điệp, nếu id đã tồn tại, nó bị loại bỏ ngay.
2. Giới hạn TTL (Time-To-Live): Trường ttl được gán giá trị ban đầu (ví dụ là 3). Mỗi lần chuyển tiếp, ttl giảm 1. Khi ttl = 0, thông điệp không được chuyển tiếp nữa, khoanh vùng phạm vi lan truyền.

b. Thuật toán Lưu và Chuyển tiếp (Store-and-Forward Routing)

- Nguyên lý: Trong mạng phân tán, nút đích có thể không thường trực (bị mất kết nối, tắt máy). Store-and-Forward cho phép các nút trung gian nhận và lưu trữ tạm thời dữ liệu, chờ đến khi đích khả dụng thì chuyển tiếp. Đây là cơ chế nền tảng của các mạng chịu trễ (Delay-Tolerant Networking – DTN).
- Lợi ích: Biến toàn bộ mạng P2P thành một hệ thống lưu trữ tạm thời, tăng đáng kể tỉ lệ tin nhắn đến được đích ngay cả trong điều kiện churn cao (peer ra vào liên tục).

c. Cơ chế Heartbeat và Phát hiện lỗi (Failure Detection)

- Nguyên lý: Trong môi trường phân tán, không thể dựa hoàn toàn vào trạng thái TCP để biết một peer còn sống hay đã chết (vì các sự cố như đứt cáp mạng, mất nguồn điện có thể khiến socket không kịp báo ngất). Heartbeat là cơ chế mà mỗi nút định kỳ gửi tín hiệu “tôi vẫn sống” (PING) đến các nút láng giềng. Nếu một nút không nhận được phản hồi (PONG) trong khoảng thời gian timeout, nó coi nút kia là đã chết và cập nhật bảng định tuyến.
- Vai trò: Đảm bảo các peer luôn có một cái nhìn nhất quán về tập hợp các peer đang hoạt động, là nền tảng cho mọi hoạt động định tuyến và chuyển tiếp tin nhắn chính xác.

Việc phối hợp các thuật toán mật mã và cơ chế phân tán tạo nên một nền tảng vững chắc cho hệ thống, giúp nó vừa an toàn trước các mối đe dọa trên mạng,

vừa có khả năng tự tổ chức và chịu đựng lỗi trong môi trường phân tán không ổn định.

2.3. Công nghệ và công cụ phát triển

Hệ thống được xây dựng dựa trên nguyên tắc hạn chế tối đa thư viện bên thứ ba, thay vào đó khai thác triệt để các module lõi (core modules) của Node.js để đảm bảo hiệu năng, tính ổn định và khả năng kiểm soát sâu đối với từng gói tin trên mạng.

Nhóm	Công nghệ	Vai trò & Lý do lựa chọn
Runtime & Core	Node.js	Môi trường thực thi JavaScript với mô hình non-blocking I/O, hướng sự kiện, cho phép một tiến trình duy nhất xử lý đồng thời hàng chục kết nối TCP mà không tốn tài nguyên thread.
Truyền thông P2P	net (TCP)	Module lõi của Node.js, cung cấp TCP socket thuần túy. Dùng để xây dựng toàn bộ kết nối trực tiếp giữa các peer (P2P Direct Connection) và kênh điều khiển đến Bootstrap Server.
Khám phá LAN	dgram (UDP)	Module lõi, cho phép gửi/nhận gói tin UDP. Dùng để thực hiện UDP Broadcast trong mạng nội bộ, giúp peer tự động phát hiện lẫn nhau mà không cần cấu hình.
Bảo mật E2EE	crypto	Module lõi, cung cấp các thuật toán mật mã: ECDH (prime256b1, secp256k1) để trao đổi khóa, AES-256-GCM để mã hóa/xác thực dữ liệu, SHA-256 để dẫn xuất khóa.
Kiến trúc module	events	Module lõi, triển khai mẫu EventEmitter (Pub/Sub) giúp các thành phần (Network, Chat, Security, UI) giao tiếp với nhau một cách lỏng lẻo, dễ bảo trì và mở rộng.
Giao diện Desktop	Electron	Framework đóng gói ứng dụng web (HTML/CSS/JS) thành ứng dụng desktop đa nền tảng. Cung cấp cơ chế IPC để giao diện (Renderer) tương tác an toàn với tiến trình xử lý mạng (Main).
Giao diện Web	Vanilla JS HTML5	Giao diện được viết thuần, không dùng framework (React/Vue). Nhẹ, render nhanh, xử lý file Base64 trực tiếp, hoàn toàn phù hợp với mục tiêu tối giản phụ thuộc.

	CSS3	
Tự động hóa & Triển khai	Power Shell / Batch Scripts	Các script .ps1, .bat được soạn sẵn để khởi động nhanh các thành phần (Bootstrap Server, Peer GUI) với các profile riêng biệt, phục vụ kiểm thử đa nút trên cùng một máy.
Hỗ trợ phát triển	Chalk (npm)	Thư viện duy nhất được thêm vào để tô màu log trên terminal, giúp lập trình viên dễ dàng phân biệt các luồng sự kiện, debug trạng thái kết nối và luồng tin nhắn.

Bảng 2.1. Công nghệ và công cụ phát triển

CHƯƠNG 3: PHÂN TÍCH HỆ THỐNG

3.1. Yêu cầu chức năng và phi chức năng

3.1.1. Yêu cầu chức năng (FR)

1. Chức năng: Tham gia mạng P2P

Mô tả: Khi khởi động, một peer phải có khả năng đăng ký sự hiện diện của mình với một thực thể đã biết (Bootstrap Server hoặc peer hiện có) để gia nhập mạng phủ. Hệ thống phải cung cấp danh sách các peer đang trực tuyến cho peer mới.

Cơ chế thực hiện:

- Peer gửi message REGISTER chứa {peerId, port} qua kết nối TCP đến Bootstrap Server tại cổng 9000.
- Bootstrap Server lưu peer vào peerRegistry và phản hồi bằng PEER_LIST chứa toàn bộ danh sách peer đang online (trừ chính nó).
- Peer nhận danh sách, cập nhật knownPeers và tự động thiết lập kết nối TCP P2P đến từng peer mới phát hiện.
- Nếu Bootstrap Server không khả dụng, peer vẫn có thể khám phá các peer khác trong LAN thông qua UDP Broadcast.

2. Chức năng: Chat trực tiếp giữa các peer

Mô tả: Người dùng có thể gửi tin nhắn văn bản trực tiếp tới một peer khác. Tin nhắn phải được truyền qua kết nối TCP P2P trực tiếp, được mã hóa đầu-cuối và có cơ chế đảm bảo tin cậy.

Cơ chế thực hiện:

- Trước khi gửi, hệ thống kiểm tra xem đã có khóa E2EE cho peer đích chưa. Nếu chưa, tự động khởi tạo quá trình trao đổi khóa ECDH.
- Nội dung được mã hóa bằng AES-256-GCM với khóa phiên dẫn xuất từ shared secret, sau đó đóng gói trong message DIRECT_CHAT.
- Message được gửi qua TCP socket trực tiếp nếu kết nối tồn tại; ngược lại, thử chuyển tiếp qua Bootstrap Server.

- Cơ chế `MessageQueue.sendWithAck()` đảm bảo gửi lại tối đa 3 lần nếu không nhận được ACK, với số thứ tự seq tăng dần.

3. Chức năng: Chat nhóm

Mô tả: Hỗ trợ chat trong một phòng (room) với nhiều peer. Tin nhắn phải được phát đến tất cả thành viên trong nhóm.

Cơ chế thực hiện:

- Peer có thể tạo hoặc tham gia một phòng qua `joinRoom(roomId)`, rời phòng qua `leaveRoom(roomId)`.
- Khi gửi tin nhắn nhóm, hệ thống duyệt danh sách thành viên của room (được duy trì trong `GroupChat.rooms`) và gửi `GROUP_CHAT` riêng lẻ đến từng thành viên qua kết nối P2P tương ứng.
- Trạng thái phòng được đồng bộ tự động bằng message `ROOM_JOIN/ROOM_LEAVE` broadcast đến toàn bộ peer.
- Nếu đã có khóa E2EE với thành viên, tin nhắn nhóm được mã hóa; ngược lại, nó được gửi ở dạng plaintext (với cảnh báo bảo mật).

4. Chức năng: Peer Discovery

Mô tả: Peer mới tham gia phải có khả năng tự động tìm và kết nối đến các peer khác trong mạng mà không cần cấu hình thủ công.

Cơ chế thực hiện: Hệ thống triển khai chiến lược khám phá đa lớp:

1. UDP Broadcast (LAN): Peer định kỳ 3 giây gửi message `DISCOVERY` chứa `{peerId, tcpPort}` đến địa chỉ broadcast `255.255.255.255:9001`. Các peer cùng mạng con nhận được và thông báo cho Peer Core.
2. Bootstrap Server Sync (WAN): Peer nhận danh sách `PEER_LIST` từ Bootstrap Server ngay sau khi đăng ký và mỗi khi có thay đổi, từ đó chủ động thiết lập kết nối TCP P2P đến các peer mới.
3. TCP Implicit Handshake: Khi nhận một kết nối TCP từ một peer chưa xác định, `TCPHandler` tự động học `peerId` từ trường `from` của message đầu tiên trao đổi qua socket.

5. Chức năng: Trạng thái online/offline

Mô tả: Hệ thống phải hiển thị danh sách các peer đang online và cập nhật tức thời khi có peer tham gia hoặc rời mạng.

Cơ chế thực hiện:

- Heartbeat (PING/PONG): Cứ mỗi HEARTBEAT_INTERVAL (3 giây), peer gửi PING đến tất cả kết nối TCP P2P đang hoạt động. Peer nhận phản hồi bằng PONG.
- Stale Peer Detection: Một timer quét định kỳ 5 giây (chu kỳ test) kiểm tra peerTimestamps. Nếu một peer không có bất kỳ tương tác nào trong HEARTBEAT_TIMEOUT (10 giây), nó bị coi là offline và bị xóa khỏi danh sách.
- Graceful Shutdown: Khi peer chủ động ngắt, nó gửi LEAVE đến tất cả peer đang kết nối để cập nhật trạng thái ngay lập tức.
- Peer phát ra sự kiện peer-discovered và peer-disconnected để lớp UI cập nhật giao diện.

6. Chức năng: Truyền tin đáng tin cậy

Mô tả: Tin nhắn phải được truyền thành công đến peer đích. Phải có cơ chế xử lý khi peer không phản hồi hoặc mất kết nối.

Cơ chế thực hiện - ba lớp đảm bảo độ tin cậy:

1. ACK/Retry (ARQ): Mỗi message DIRECT_CHAT được gán seq tăng dần. Bên nhận gửi lại ACK chứa seq. Nếu sau ACK_TIMEOUT (5 giây) không nhận được ACK, message được gửi lại, tối đa MAX_RETRIES (3 lần). Logic này được quản lý bởi MessageQueue.
2. Deduplication: Mỗi message có id duy nhất. Peer duy trì seenMessageIds (vòng đệm 2000 ID gần nhất) để loại bỏ message đến trùng lặp.
3. Store-and-Forward: Khi peer đích offline, message được đóng gói thành STORE_AND_FORWARD và gửi đến tất cả peer khác. Các peer trung gian lưu vào relayQueue và tự động chuyển tiếp khi peer đích online trở lại (sự kiện connection-established).

7. Chức năng nâng cao

Hệ thống triển khai đầy đủ các chức năng nâng cao theo mục tiêu của đề án:

- Broadcast toàn mạng (Global Chat)
- Store-and-Forward messaging
- Mã hóa tin nhắn E2EE (ECDH + AES-256-GCM)
- Giao diện GUI (Electron)
- Truyền file trực tiếp (Base64, <5MB)
- Mô phỏng churn (peer join/leave liên tục)

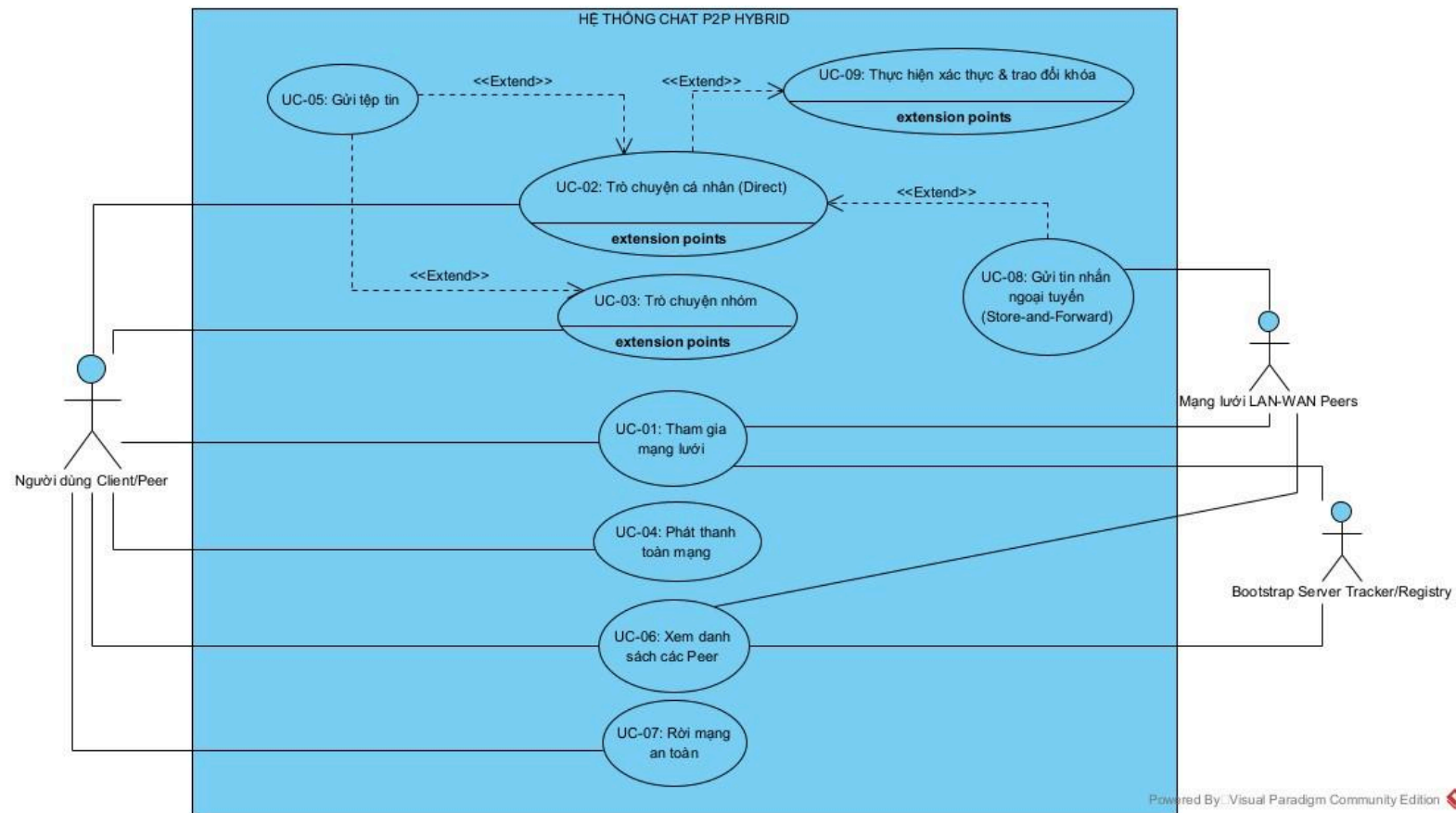
3.1.2. Yêu cầu phi chức năng (NFR)

ID	Yêu cầu	Chỉ tiêu đo lường	Phương pháp đánh giá	Mức độ
01	Hiệu năng	Độ trễ < 50ms (LAN), < 200ms (WAN)	Đo thời gian send→receive qua timestamp	High
02	Bảo mật	Không lưu plaintext; key exchange an toàn; mã hóa AES-256-GCM	Audit code + Wireshark packet inspection	Critical
03	Khả dụng	Hệ thống tiếp tục hoạt động khi Bootstrap Server offline (với peer đã biết)	Test scenario ngắt server đột ngột	Medium
04	Khả năng mở rộng	Hỗ trợ ≥ 50 peer đồng thời	Load test với churn-test.js	Medium
05	Khả chuyển	Hoạt động trên Windows với Node.js 18+	Cross-platform testing matrix	High
06	Độ tin cậy	Tỷ lệ gửi tin thành công > 95%	Thống kê ACK/Retry trong test suite	High
07	Khả năng phục hồi	Tự động reconnect Bootstrap Server sau 3 giây khi mất kết nối	Test ngắt mạng và khôi phục	Medium

Bảng 3.1. Bảng yêu cầu phi chức năng

3.2. Sơ đồ Use Case

3.2.1. Sơ đồ tổng quát



Hình 3.1. Sơ đồ Use Case tổng quát của hệ thống Chat P2P Hybrid

3.2.2. Nhận diện Tác nhân (Actor)

Hệ thống xác định hai nhóm tác nhân chính có khả năng tương tác và ảnh hưởng đến hoạt động của mạng P2P:

a. Tác nhân chính (Primary Actor)

Người dùng (Client/Peer): Là người trực tiếp khởi chạy ứng dụng Electron và thao tác trên giao diện đồ họa. Người dùng có thể gửi tin nhắn văn bản, gửi file, tạo/tham gia phòng chat, xem danh sách peer đang online và chủ động ngắt kết nối. Mỗi hành động của người dùng sẽ kích hoạt các luồng xử lý phức tạp bên dưới nhưng được hệ thống che giấu hoàn toàn.

b. Tác nhân hệ thống (System Actors)

Bootstrap Server (Tracker/Registry): Máy chủ trung tâm nhẹ, không tham gia truyền dữ liệu chat, chỉ đóng vai trò "danh bạ". Nó tiếp nhận đăng ký từ các peer, duy trì danh sách peer online và cung cấp danh sách này cho peer mới tham gia. Trong một số tình huống, nó còn hoạt động như một relay dự phòng cho các message điều khiển.

Mạng lưới LAN/WAN Peers: Tập hợp các máy tính khác đang chạy cùng ứng dụng P2P. Chúng tương tác với peer hiện tại theo hai vai trò:

- Đối tác khám phá (Discovery Peers): Phản hồi các gói tin UDP Broadcast trong LAN hoặc xuất hiện trong danh sách PEER_LIST từ Bootstrap Server, giúp peer mới mở rộng mạng lưới.
- Trạm trung chuyển (Relay Peers): Khi một peer đích offline, các peer đang online khác đóng vai trò lưu trữ tạm tin nhắn (Store) và tự động chuyển tiếp (Forward) khi đích trực tuyến trở lại.

3.2.3. Đặc tả Use Case chi tiết (Use Case Specifications)

a. UC-01: Tham gia mạng P2P

Tên Use Case	Tham gia mạng P2P
Actor	- Người dùng (Client/Peer) - Bootstrap Server, LAN Peers

Tiền điều kiện	Ứng dụng Electron được khởi chạy thành công; Máy tính của người dùng có kết nối mạng (LAN hoặc Internet).
Hậu điều kiện	Peer tích hợp thành công vào mạng lưới; bảng định tuyến cục bộ (knownPeers) được cập nhật; trạng thái của Peer hiển thị "Online" đối với toàn mạng.
Luồng chính	1. Người dùng kích hoạt khởi chạy ứng dụng Chat P2P. 2. Hệ thống tự động mở các cổng mạng cục bộ để chuẩn bị lắng nghe kết nối. 3. Hệ thống phát tín hiệu dò tìm (Broadcast) tới các máy tính trong cùng mạng nội bộ (LAN). 4. Đồng thời, hệ thống thiết lập kết nối tới Máy chủ trung tâm (Bootstrap Server) trên Internet. 5. Hệ thống gửi thông tin định danh (Gói tin REGISTER) lên Máy chủ trung tâm. 6. Máy chủ trung tâm ghi nhận thông tin, phản hồi lại danh sách các Peer đang hoạt động (PEER_LIST) cho hệ thống. 7. Máy chủ trung tâm phát thông báo đến các Peer khác trên mạng để báo hiệu có Node mới vừa tham gia. 8. Hệ thống tổng hợp và hiển thị danh sách toàn bộ các Peer tìm thấy lên thanh điều hướng (Sidebar) cho người dùng.
Ngoại lệ	4.1 (Mất kết nối Internet hoàn toàn): Tại bước 4, kết nối tới Bootstrap Server thất bại. Ứng dụng chuyển sang chế độ hoạt động ngoại tuyến cục bộ (LAN Mode), chỉ thực hiện luồng Broadcast ở bước 3 để giao tiếp với các máy trong cùng mạng nội bộ. 6.1 (Trùng lặp PeerID): Tại bước 6, Bootstrap Server phát hiện ID này đã tồn tại trong Registry. Server phản hồi mã lỗi từ chối, ứng dụng tự động đóng ngắt kết nối an toàn và yêu cầu người dùng đổi định danh cấu hình profile.

b. UC-02: Trò chuyện cá nhân 1-1

Tên Use Case	Trò chuyện cá nhân 1-1
Actor	- Người dùng (Client/Peer) - LAN Peers
Tiền điều kiện	Đối phương nằm trong danh bạ mạng lưới của người gửi; cả hai đều đang hoạt động trực tuyến (Online).

Hậu điều kiện	Tin nhắn được truyền tải an toàn; bên nhận giải mã hiển thị thành công; hoặc tin nhắn được ký gửi thành công vào hàng đợi ngoại tuyến của mạng lưới.
Luồng chính	1. Người dùng chọn một tài khoản bạn bè trên giao diện ứng dụng để mở phiên chat. 2. [Bao gồm - Include UC-09]: Hệ thống tự động kiểm tra và thực hiện tiến trình Trao đổi khóa bảo mật (ECDH Handshake) ngầm với đối phương để sinh ra Khóa bí mật chung. 3. Người dùng nhập nội dung thông điệp và bấm nút "Gửi". 4. Hệ thống áp dụng thuật toán mã hóa (AES-GCM) để khóa hoàn toàn nội dung thông điệp bằng Khóa bí mật chung. 5. Hệ thống đóng gói và truyền tin nhắn mã hóa qua đường truyền trực tiếp (Direct Connection) tới máy đối phương. 6. Máy đối phương tiếp nhận gói tin, xác thực tính toàn vẹn, giải mã nội dung và hiển thị lên khung chat. 7. Máy đối phương tự động dội lại một tín hiệu Xác nhận (ACK). 8. Hệ thống của người gửi nhận tín hiệu ACK và cập nhật trạng thái "Đã gửi / Đã nhận" lên giao diện.
Ngoại lệ	5.1 (Đối phương đột ngột ngoại tuyến): Việc thiết lập liên kết trực tiếp bị thất bại (do đối phương mất mạng, tắt máy ngang): - Hệ thống tự động kích hoạt [Mở rộng - Extend UC-08: Store-and-Forward]. - Gói tin mã hóa được hệ thống chuyển hướng, ký gửi tạm thời lên các trạm Relay Peer trực tuyến khác để nhờ lưu và chuyển tiếp hộ khi đối phương quay lại mạng. - Giao diện cập nhật trạng thái: "<i>Đang chờ gửi...</i>".

c. UC-03: Trò chuyện nhóm

Tên Use Case	Trò chuyện nhóm
Actor	- Người dùng (Client/Peer)
Tiền điều kiện	Phòng chat đã được khởi tạo trên mạng lưới. Người gửi đã hoàn thành thủ tục tham gia phòng và đang nắm giữ Mã khóa phòng bí mật (Room Key).
Hậu điều kiện	Toàn bộ các thành viên đang hoạt động trong phòng đều nhận và giải mã hiển thị chính xác nội dung tin nhắn. Các thành viên

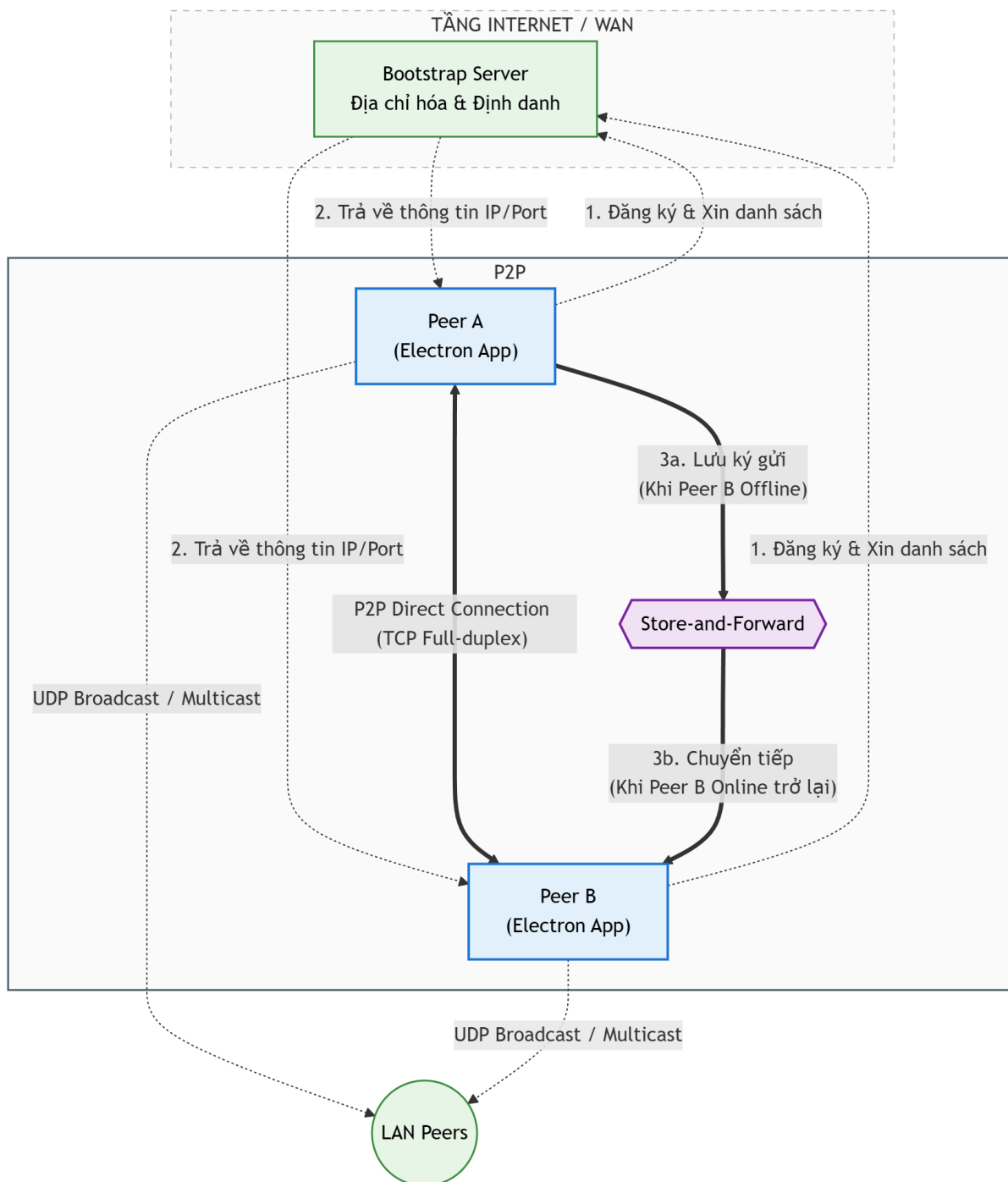
	ngoại tuyến được bảo đảm nhận tin sau thông qua cơ chế dự phòng
Luồng chính	1. Người dùng truy cập vào một phòng trò chuyện nhóm hiện có trên ứng dụng. 2. Người dùng nhập nội dung thông điệp và bấm "Gửi". 3. Hệ thống tự động truy xuất Mã khóa phòng bí mật đang được lưu trữ an toàn tại máy trạm cục bộ. 4. Hệ thống áp dụng thuật toán mã hóa (AES-GCM) sử dụng Mã khóa phòng này để bảo vệ nội dung thông điệp. 5. Hệ thống gắn định danh mã phòng (Room ID) vào vỏ gói tin để phục vụ định tuyến. 6. Hệ thống tự động duyệt qua danh sách tất cả các thành viên thuộc phòng đó đang có trạng thái Online. 7. Hệ thống thực hiện phát tán gói tin song song qua đường truyền mạng trực tiếp (Direct Connection) tới từng thành viên trong danh sách. 8. Hệ thống của các thành viên tiếp nhận gói tin, sử dụng Mã khóa phòng tương ứng đang giữ để giải mã và hiển thị thông điệp lên giao diện trò chuyện.
Ngoại lệ	2.1 (Vi phạm phân quyền):hệ thống phát hiện người dùng chưa thực hiện thủ tục tham gia phòng (Chưa có Mã khóa phòng). Giao diện lập tức chặn hành động gửi và đưa ra thông báo cảnh báo lỗi phân quyền hệ thống. 7.1 (Thành viên nhóm ngoại tuyến đột ngột):đối với những thành viên không thể kết nối trực tiếp (đã tắt máy / rút mạng): + Hệ thống bỏ qua đường truyền trực tiếp cho thành viên đó. + Tự động kích hoạt [Mở rộng - Extend UC-08: Store-and-Forward], đưa gói tin vào hàng đợi ký gửi tại các trạm trung gian để đồng bộ lại khi thành viên đó kết nối trở lại.

c. UC-08: Gửi tin nhắn ngoại tuyến (Store-and-Forward)

Tên Use Case	Gửi tin nhắn ngoại tuyến
Actor	- Người dùng (Client/Peer) - LAN Peers
Tiền điều kiện	Mất kết nối mạng trực tiếp giữa máy gửi và máy đích. Mạng lưới P2P hiện đang có ít nhất một máy trung gian (Relay) đang hoạt động.

Hậu điều kiện	Tin nhắn đến tay người nhận an toàn tuyệt đối mà không hề bị lộ nội dung rõ (Plaintext) tại bất kỳ trạm trung gian nào
Luồng chính	[Giai đoạn 1: Ký gửi tin nhắn - Store] 1. Người dùng tiến hành gửi tin nhắn cho bạn bè. 2. Hệ thống của máy gửi phát hiện liên kết mạng tới máy đích bị gián đoạn (Offline). 3. Thay vì hủy bỏ, hệ thống chuyển hướng gói tin mã hóa vào hàng đợi ký gửi. 4. Hệ thống tự động tìm kiếm một máy trung chuyển (Relay Peer - Node C) đang Online trong mạng và gửi gói tin nhờ lưu hộ (kèm theo nhãn ID của máy đích). 5. Hệ thống của Node C tiếp nhận gói tin, lưu trữ vào bộ nhớ đệm (RAM) cục bộ và phản hồi tín hiệu xác nhận (ACK). 6. Giao diện máy gửi hiển thị trạng thái "Đang chờ đối phương online". [Giai đoạn 2: Bàn giao chuyển tiếp - Forward] 7. Máy đích (Peer B) khởi động lại ứng dụng và khai báo trạng thái trực tuyến lên Máy chủ trung tâm (Bootstrap Server). 8. Máy chủ trung tâm phát sóng tín hiệu Online của Peer B tới toàn mạng (bao gồm cả Node C). 9. Hệ thống của Node C nhận diện Peer B đã quay lại, lập tức giải phóng hàng đợi tin nhắn đệm. 10. Hệ thống của Node C chủ động kết nối tới Peer B và tự động đẩy toàn bộ các bản tin đang giữ hộ sang. 11. Máy đích (Peer B) tiếp nhận gói tin, sử dụng khóa mật mã cục bộ của mình để giải mã hiển thị thông điệp thành công.
Ngoại lệ	4.1 (Mạng lưới cô lập hoàn toàn): Hệ thống không tìm thấy bất kỳ máy trung gian nào đang hoạt động, gói tin sẽ được giữ lại trong bộ nhớ tạm của chính máy gửi. Hệ thống tự động kích hoạt cơ chế thử lại (Retry) định kỳ cho đến khi có một máy trung gian xuất hiện. - Ngoại lệ 2 (Trạm trung gian sập nguồn đột ngột): Trong thời gian chờ Peer B online, nếu Node C bị mất điện/tắt ứng dụng, toàn bộ dữ liệu lưu trên bộ nhớ đệm (RAM) của Node C sẽ bị mất. Tuy nhiên, tính toàn vẹn của thông tin vẫn được đảm bảo nhờ cơ chế Retry ở Ngoại lệ 1 (Máy gửi ban đầu vẫn giữ bản sao và sẽ nhờ một Node D khác lưu hộ).

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG & CẤU TRÚC DỮ LIỆU



Hình 4.1. Sơ đồ kiến trúc tổng thể của hệ thống Chat P2P Hybrid

4.1. Kiến trúc hệ thống phân lớp

Hệ thống được thiết kế theo mô hình Hướng sự kiện (Event-Driven) kết hợp với Kiến trúc Phân lớp (Layered Architecture). Việc phân lớp giúp hệ thống tuân thủ nguyên lý Đơn trách nhiệm (Single Responsibility Principle), cô lập sự phức tạp của giao thức mạng khỏi logic nghiệp vụ của ứng dụng.

4.1.1. Mô hình bốn lớp chức năng

Hệ thống được chia thành bốn lớp chính xếp chồng từ thấp đến cao:

1. Lớp Giao tiếp mạng (Network Layer): Tầng thấp nhất, tiếp xúc trực tiếp với hệ điều hành thông qua socket. Bao gồm:

- TCPHandler: Quản lý các kết nối TCP P2P trực tiếp, lắng nghe và chấp nhận kết nối đến, gửi/nhận dữ liệu dưới dạng newline-delimited JSON, thực hiện nhận dạng ẩn (implicit handshake) qua trường from.
- UDPHandler: Phát và lắng nghe gói tin UDP broadcast trong mạng LAN để khám phá peer mà không cần Internet.
- BootstrapClient: Duy trì kênh TCP đến Bootstrap Server, gửi REGISTER, nhận PEER_LIST và hỗ trợ relay khi cần.

2. Lớp An toàn bảo mật (Security Layer): Đảm bảo tính riêng tư và toàn vẹn dữ liệu. Bao gồm:

- KeyExchange: Sinh cặp khóa ECDH, thực hiện bắt tay trao đổi khóa với các peer khác, lưu trữ an toàn sharedSecret cho từng peer.
- Crypto: Cung cấp các hàm tính mã hóa/giải mã AES-256-GCM và dẫn xuất khóa SHA-256. Đây là lớp tiện ích không trạng thái, được các thành phần khác sử dụng.

3. Lớp Lõi điều phối (Core Layer): “Trái tim” của hệ thống, đại diện bởi lớp Peer. Nó hoạt động như một Facade che giấu toàn bộ sự phức tạp bên dưới. Peer chịu trách nhiệm:

- Quản lý vòng đời của tất cả các handler (khởi tạo, khởi động, dừng).
- Điều phối message đến đúng handler (DirectChat, GroupChat, v.v.).

1-1 mà không cần quan tâm socket vật lý. Crypto là lớp tiện ích không trạng thái, chỉ chứa các hàm xử lý mật mã.

- Kiến trúc Facade: Lớp Peer đóng vai trò mặt tiền duy nhất cho UI. UI gọi `peer.sendToPeer(...)` hoặc nhận sự kiện từ Peer, không cần biết bên trong có bao nhiêu handler. Nhờ vậy, nếu thay đổi cơ chế gửi (thêm relay mới, đổi thuật toán mã hóa), UI không bị ảnh hưởng.
- Quan hệ Composition (owns): Các lớp `TCPHandler`, `UDPHandler`, `BootstrapClient`, `KeyExchange`, `StoreAndForward_MessageQueue` được Peer khởi tạo và sở hữu duy nhất (ký hiệu *-- với hình thoi đặc). Vòng đời của chúng gắn liền với Peer: khi Peer bị hủy, tất cả kết nối mạng và khóa được dọn dẹp, tránh rò rỉ tài nguyên.
- Quan hệ Dependency (uses): `DirectChat`, `GroupChat` và `KeyExchange` phụ thuộc vào `Crypto` (mũi tên đứt) để sử dụng các hàm mã hóa tĩnh. `DirectChat` và `GroupChat` cũng phụ thuộc vào `KeyExchange` để lấy khóa phiên, thể hiện sự kết nối giữa tầng chat và tầng bảo mật.

4.2. Thiết kế Giao thức trao đổi thông điệp (Message Protocol)

4.2.1. Cấu trúc khung truyền tải JSON-over-TCP

Hệ thống lựa chọn TCP socket thuần (raw TCP) thay vì HTTP hay WebSocket vì các lý do:

- Hiệu năng và kiểm soát: Raw TCP cho phép gửi nhận dữ liệu dưới dạng luồng byte liên tục với độ trễ thấp nhất, không chịu overhead của HTTP headers. Phù hợp với mô hình P2P nơi dữ liệu được trao đổi liên tục và hai chiều độc lập.
- Tính linh hoạt: Không bị ràng buộc bởi mô hình request-response của HTTP. Có thể gửi message bất kỳ lúc nào, đúng với bản chất của chat thời gian thực.
- Sự đơn giản: JSON là định dạng con người đọc được, dễ debug, được hỗ trợ sẵn trong JavaScript. Kết hợp với TCP, ta có một kênh truyền tin cậy, có thứ tự, không cần tự xây dựng lớp kiểm soát luồng.

Xử lý vấn đề phân mảnh (Sticky Packets): TCP là luồng byte không có ranh giới message. Khi gửi liên tiếp nhiều message, chúng có thể bị dính vào nhau trong

buffer của bên nhận. Để giải quyết, hệ thống sử dụng kỹ thuật newline-delimited JSON:

- Mỗi message JSON được nối thêm ký tự xuống dòng ‘\n’ (0x0A) làm dấu phân cách.
- Bên nhận duy trì một buffer chuỗi. Khi nhận được chunk mới, nối vào buffer, sau đó liên tục tách theo ‘\n’. Những phần chưa có ‘\n’ được giữ lại chờ chunk tiếp theo.
- Cách làm này đảm bảo message luôn được xử lý trọn vẹn, không lo dính gói hay mất dữ liệu.

Dưới đây là đoạn Code trích xuất mã nguồn xử lý luồng sự kiện socket (_setupSocketEvents) tại lớp TCPHandler trong dự án:

```
socket.on('data', (chunk) => {
    buffer += chunk.toString();
    // 1. Tích lũy luồng byte vào bộ đệm
    let boundary = buffer.indexOf('\n');
    // 2. Tìm ranh giới của gói tin
    while (boundary !== -1) {
    // Cắt lấy thông điệp nguyên vẹn
        const msgStr = buffer.substring(0, boundary);
    // Cập nhật lại bộ đệm cho các byte còn thừa
        buffer = buffer.substring(boundary + 1);
        boundary = buffer.indexOf('\n');
        try {
            const msg = JSON.parse(msgStr);
    // 3. Parse chuỗi thành Object JSON
            // ... (xử lý logic phía dưới)
        } catch (e) {
            // Bỏ qua gói tin bị lỗi định dạng
        }
    }
});
```

4.2.2. Đặc tả Bảng mã thông điệp (Message Types)

Hệ thống định nghĩa tập hợp các loại thông điệp với cấu trúc JSON rõ ràng, được liệt kê:

Loại (type)	Hướng	Mô tả	Cấu trúc payload chính
-------------	-------	-------	------------------------

REGISTER	Peer → Bootstrap	Đăng ký sự hiện diện	{ "type": "REGISTER", "from": "peerA", "payload": { "port": 52341 } }
PEER_LIST	Bootstrap → Peer	Cấp danh sách peer online	{ "type": "PEER_LIST", "from": "bootstrap", "payload": { "peers": [{ "id": "peerB", "ip": "...", "port": ... }] } }
DIRECT_ CHAT	Peer ↔ Peer	Tin nhắn riêng tư (đã mã hóa)	{ "type": "DIRECT_CHAT", "from": "peerA", "to": "peerB", "id": "...", "seq": 42, "payload": { "encrypted": { "iv": "...", "content": "...", "tag": "... } } }
GROUP_ CHAT	Peer ↔ Peer	Tin nhắn trong phòng	{ "type": "GROUP_CHAT", "from": "peerA", "to": "room1", "payload": { "roomId": "room1", "encrypted": { ... } } }
GLOBAL_ CHAT	Peer → All	Broadcast toàn mạng	{ "type": "GLOBAL_CHAT", "from": "peerA", "ttl": 3, "payload": { "encrypted": { ... } } }
KEY_ EXCHANGE_ INIT	Peer → Peer	Gửi public key ECDH	{ "type": "KEY_EXCHANGE_INIT", "from": "peerA", "to": "peerB", "payload": { "publicKey": "...", "curve": "prime256v1" } }
KEY_ EXCHANGE_ RESPONSE	Peer → Peer	Phản hồi public key	{ "type": "KEY_EXCHANGE_RESPONSE", "from": "peerB", "to": "peerA", "payload": { "publicKey": "... } }

ACK	Peer → Peer	Xác nhận đã nhận message	{ "type": "ACK", "from": "peerB", "to": "peerA", "payload": { "seq": 42, "status": "ok" } }
STORE_ AND_ FORWARD	Peer → Relay	Nhờ lưu và chuyển tiếp	{ "type": "STORE_AND_FORWARD", "from": "peerA", "payload": { "target": "peerB", "message": { ... }, "ttl": 3600000 } }
PING / PONG	Peer ↔ Peer	Heartbeat giữ kết nối	{ "type": "PING", "from": "peerA" }
LEAVE	Peer → Peer	Thông báo rời mạng	{ "type": "LEAVE", "from": "peerA" }

Bảng 4.1. Đặc tả các loại thông điệp

Mỗi message đều có thể chứa thêm timestamp và id để chống trùng lặp và theo dõi.

4.2.3. Tích hợp Mã hóa đầu cuối (E2EE) vào gói tin

Mã hóa được tích hợp ngay trong cấu trúc payload, chỉ bảo vệ phần nội dung cần giữ bí mật, trong khi các trường metadata (định tuyến, loại) vẫn ở dạng rõ để các nút trung gian (Bootstrap, Relay) có thể xử lý. Cơ chế này được gọi là mã hóa một phần (partial encryption).

- Khi gửi DIRECT_CHAT, plaintext được mã hóa thành một object gồm {iv, content, tag} bằng AES-256-GCM, sử dụng khóa dẫn xuất từ sharedSecret. Object này được đặt vào payload.encrypted.
- Các trường type, from, to, seq, id, timestamp không bị mã hóa, cho phép hệ thống định tuyến và kiểm soát tin nhắn mà không cần giải mã.
- Với tin nhắn nhóm (GROUP_CHAT), tương tự, roomId nằm ngoài phần mã hóa để biết message thuộc phòng nào.
- Khi Store-and-Forward, Relay Peer chỉ lưu giữ toàn bộ message đã mã hóa, không bao giờ có khóa giải mã, đảm bảo tính riêng tư ngay cả khi tin nhắn được lưu tạm trên máy khác.

Minh họa cấu trúc payload của DIRECT_CHAT:

```
{
  "type": "DIRECT_CHAT",
  "from": "PeerA_12345",
  "to": "PeerB_67890",
  "id": "msg_9999",
  "seq": 42,
  "timestamp": 1716800000000,
  "payload": {
    "encrypted": {
      "iv": "3a7b9c...",
      "content": "8f1a2e...",
      "tag": "c5d8e9..."
    }
  }
}
```

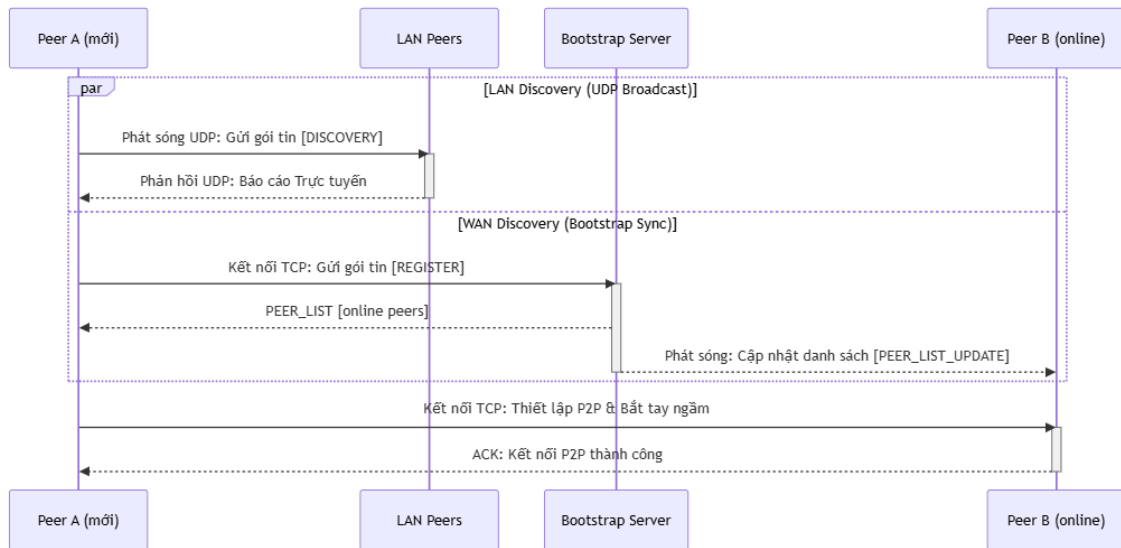
Như vậy, giao thức đảm bảo chỉ người sở hữu `sharedSecret` mới đọc được nội dung, nhưng vẫn cho phép các node trung gian thực hiện đúng chức năng mạng.

4.3. Thiết kế Cơ chế Vận hành và Định tuyến

Sức mạnh của hệ thống Hybrid P2P nằm ở khả năng tự động tổ chức và dệt mạng lưới (Self-organizing mesh) mà không cần sự can thiệp của người dùng. Quá trình này được thực hiện thông qua hai cơ chế cốt lõi: Khám phá mạng lưới và Thỏa thuận khóa bảo mật.

4.3.1. Khám phá mạng lưới (Peer Discovery)

Để đảm bảo tính sẵn sàng cao nhất (High Availability), hệ thống được thiết kế với 3 cơ chế dò tìm chạy song song, cho phép các nút mạng (Node) tìm thấy nhau trong mọi điều kiện hạ tầng:



Hình 4.3. Sơ đồ tuần tự khám phá peer (LAN và WAN)

1. Cơ chế: Khám phá Cục bộ (LAN Discovery - Offline Mode) - Phát sóng UDP Broadcast trong LAN:

- Mục đích: Đảm bảo hệ thống vẫn hoạt động nhắn tin nội bộ bình thường ngay cả khi mạng Internet (WAN) bị gián đoạn.
- Hoạt động: Cứ mỗi 3 giây (theo cấu hình hằng số `UDP_BROADCAST_INTERVAL`), lớp `UDPHandler` sẽ phát một gói tin `DISCOVERY` chứa { `peerId`, `tcpPort` } bằng sóng vô tuyến tới địa chỉ broadcast `255.255.255.255:9001`. Các Peer trong cùng mạng LAN khi nhận được sóng này sẽ tự động trích xuất địa chỉ IP trực tiếp từ nguồn gửi của gói tin (không cần gửi kèm IP trong payload) và cập nhật vào danh bạ nội bộ.

2. Cơ chế: Khám phá Diện rộng (WAN Discovery - Online Mode) - Đồng bộ danh bạ qua Bootstrap Server trong WAN:

- Mục đích: Kết nối các Peer ở các vị trí địa lý khác nhau (khác dải mạng LAN) thông qua Internet.
- Hoạt động: Khi khởi động, Lớp `BootstrapClient` gửi gói tin `REGISTER` tới Máy chủ trung tâm (Bootstrap Server). Máy chủ ghi nhận và phản hồi lại gói `PEER_LIST` chứa danh sách mạng lưới hiện tại.
- Đồng bộ tức thời: Mỗi khi có một Peer mới gia nhập hoặc rời mạng (ngắt kết nối), Máy chủ trung tâm sẽ loại bỏ/thêm mới Peer đó khỏi Registry và

chủ động phát sóng lại toàn bộ danh sách PEER_LIST mới nhất tới tất cả các Peer đang online, đảm bảo danh bạ của toàn mạng luôn được đồng bộ tức thời.

3. Cơ chế: "Học danh tính ngầm" (Implicit Peer Identification)

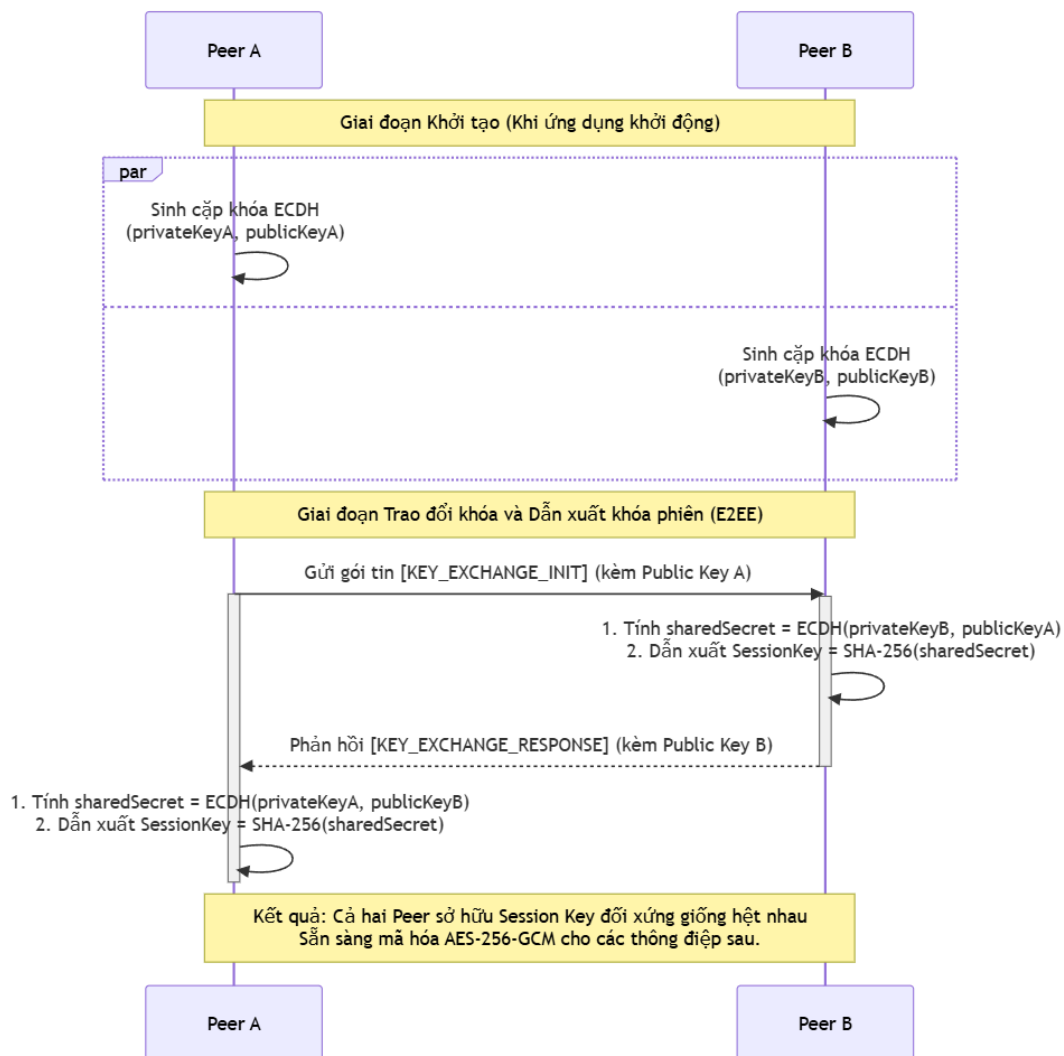
- Mục đích: Tối ưu hóa quá trình tạo mạng lưới P2P, giảm thiểu số lượng gói tin bắt tay thừa thãi.
- Hoạt động: Khi Peer A chủ động thiết lập kết nối TCP tới Peer B, hệ thống của Peer B (đang ở thể bị động) sẽ trích xuất định danh từ trường msg.from của gói tin TCP đầu tiên. Nếu nhận thấy ID này chưa tồn tại trong danh bạ, Peer B sẽ tự động học danh tính này, lưu trữ lại đối tượng Socket và thiết lập liên kết song công. Cơ chế này giúp mạng lưới tự động nhận diện nhau (Self-healing) vô cùng mượt mà.

4.3.2. Tiến trình Thỏa thuận khóa bảo mật (ECDH Handshake)

Nguyên tắc tối thượng của hệ thống là: “Không có khóa thì không truyền dữ liệu”. Nhằm tối ưu hóa hiệu năng, mỗi Peer chỉ sinh ra một cặp khóa đường cong Elliptic duy nhất (sử dụng chuẩn prime256v1 hoặc secp256k1) vào thời điểm khởi động ứng dụng (trong constructor của KeyExchange). Cặp khóa này sẽ được tái sử dụng cho mọi phiên bắt tay:

1. Khởi tạo (Initiate): Khi Peer A muốn chat với Peer B nhưng kiểm tra thấy chưa có khóa bí mật chung (Shared Secret), Peer A sẽ lấy Public Key từ cặp khóa đã sinh sẵn của mình, đóng gói vào thông điệp KEY_EXCHANGE_INIT và gửi trực tiếp qua TCP Socket tới Peer B.
2. Đáp trả (Response): Peer B nhận được thông điệp. Ngay lập tức, lớp KeyExchange của Peer B dùng Private Key của chính mình kết hợp với Public Key của A vừa nhận để tính toán ra Khóa bí mật chung (Shared Secret). Sau đó, Peer B lấy Public Key của mình đóng gói vào gói KEY_EXCHANGE_RESPONSE trả lại cho Peer A.
3. Hoàn tất (Finalize): Peer A nhận được Public Key của B, thực hiện phép toán tương tự để sinh ra Khóa bí mật chung. Bằng thuật toán đường cong Elliptic, hai Khóa bí mật chung được tính toán ở hai máy độc lập sẽ hoàn toàn giống hệt nhau.

4. Khóa bí mật chung này sau đó được đẩy qua hàm băm SHA-256 để dẫn xuất ra khóa mã hóa phiên. Kể từ lúc này, mọi tin nhắn trao đổi giữa A và B sẽ được AES-256-GCM bảo vệ toàn vẹn.



Hình 4.4. Sơ đồ Tuần tự: Tiến trình trao đổi khóa ECDH

4.4. Thiết kế Cơ chế Chịu lỗi và Đảm bảo độ tin cậy (Fault Tolerance & Reliability)

Mạng ngang hàng (P2P) mang bản chất biến động rất cao (High Churn Rate). Các nút mạng (Peer) có thể tắt máy, rút mạng hoặc di chuyển liên tục mà không có sự báo trước. Để hệ thống ứng dụng Chat duy trì được tính ổn định và không bị mất mát dữ liệu, đồ án đã thiết kế và tích hợp hai cơ chế chịu lỗi cốt lõi: Khung phát hiện lỗi (Heartbeat) và Định tuyến dự phòng (Store-and-Forward).

4.4.1. Khung phát hiện lỗi (Failure Detection: Heartbeat PING/PONG)

Trong kiến trúc TCP Socket, khi một máy tính đột ngột mất điện hoặc đứt cáp mạng, tín hiệu ngắt kết nối (FIN/RST) thường không được gửi đi, dẫn đến hiện tượng kết nối "chết ảo" (Half-open connections). Lớp Lỗi (Peer) có thể vẫn lầm tưởng đối phương đang trực tuyến và tiếp tục đẩy dữ liệu vào một socket đã chết.

Để khắc phục hiện tượng này, đồ án triển khai thuật toán Heartbeat (Nhịp tim):

- Chủ động thăm dò (Active Probing): Định kỳ mỗi 3 giây (được định nghĩa bởi hằng số HEARTBEAT_INTERVAL), hệ thống sẽ gửi một gói tin trọng lượng nhẹ loại PING tới tất cả các Peer đang nằm trong danh sách kết nối hiện hoạt (activeConnections). Bất kỳ máy nào nhận được PING đều phải lập tức dội ngược lại gói tin PONG.
- Phát hiện Node chết (Stale Peer Detection): Lớp Peer duy trì cấu trúc dữ liệu bản đồ thời gian (peerTimestamps: Map). Bất cứ khi nào nhận được một tín hiệu từ đối phương, hàm handleIncomingMessage() và onPong() sẽ tự động cập nhật mốc thời gian (Timestamp) mới nhất cho Peer đó.
- Đào thải (Eviction): Hệ thống liên tục kiểm tra bản đồ thời gian này. Nếu một Peer không có bất kỳ tương tác nào vượt quá 10 giây (ngưỡng HEARTBEAT_TIMEOUT), nó lập tức bị coi là ngoại tuyến (Offline). Hệ thống chủ động đóng socket, xóa Peer khỏi bảng định tuyến (knownPeers) để tránh rò rỉ tài nguyên. Việc cấu hình timeout ở mức độ giây giúp hệ thống phản ứng với các rủi ro sập mạng gần như theo thời gian thực.

4.4.2 Thiết kế hàng đợi & Định tuyến ngoại tuyến (Store-and-Forward)

Trong trường hợp người dùng cần gửi tin nhắn quan trọng nhưng đối phương lại đang ngoại tuyến (hoặc vừa bị đào thải bởi thuật toán Heartbeat ở trên), hệ thống không cho phép hủy bỏ gói tin. Thay vào đó, một cơ chế định tuyến dự phòng được kích hoạt thông qua lớp StoreAndForward_MessageQueue. Cơ chế này hoạt động theo 3 pha nhằm đảm bảo tính toàn vẹn dữ liệu:

a. Cơ chế Thử lại (Retry) & Xác nhận (ACK):

- Mỗi tin nhắn gửi đi đều được đính kèm một số thứ tự (Sequence Number). Hàm `MessageQueue.sendWithAck()` chịu trách nhiệm theo dõi gói tin này.
- Nếu không nhận được tín hiệu xác nhận (ACK) từ máy đích sau 5 giây (ACK_TIMEOUT), hệ thống sẽ tự động thử gửi lại (Retry). Sau khi thử lại tối đa 3 lần (MAX_RETRIES) mà vẫn thất bại, hệ thống chính thức chuyển gói tin sang chế độ ngoại tuyến.

b. Pha Ký gửi (Store):

- Gói tin (đã được mã hóa E2EE bằng AES-256-GCM) được chuyển hướng, đóng gói vào lớp vỏ `STORE_AND_FORWARD` kèm nhãn thông tin đích (target: `PeerB_ID`).
- Hệ thống chủ động gửi gói này sang một trạm lân cận đang Online (Relay Peer - Node C) để nhờ giữ hộ. Node C chỉ lưu trữ bản tin này trên bộ nhớ đệm (RAM) mà không có khóa để đọc nội dung, đảm bảo tuyệt đối tính riêng tư.

c. Pha Bàn giao (Forward):

- Khi người nhận (Peer B) quay trở lại mạng lưới, hệ thống sẽ phát tín hiệu trực tuyến thông qua cơ chế đồng bộ `PEER_LIST` của Bootstrap Server.
- Ngay khi một kết nối TCP giữa Node C và Peer B được thiết lập trở lại (bất kể do Peer B chủ động gọi tới hay Node C chủ động gọi đi, thông qua sự kiện `connection-established`), Node C sẽ tự động duyệt hàng đợi và chuyển tiếp (Forward) toàn bộ tin nhắn đang giữ hộ sang cho Peer B.

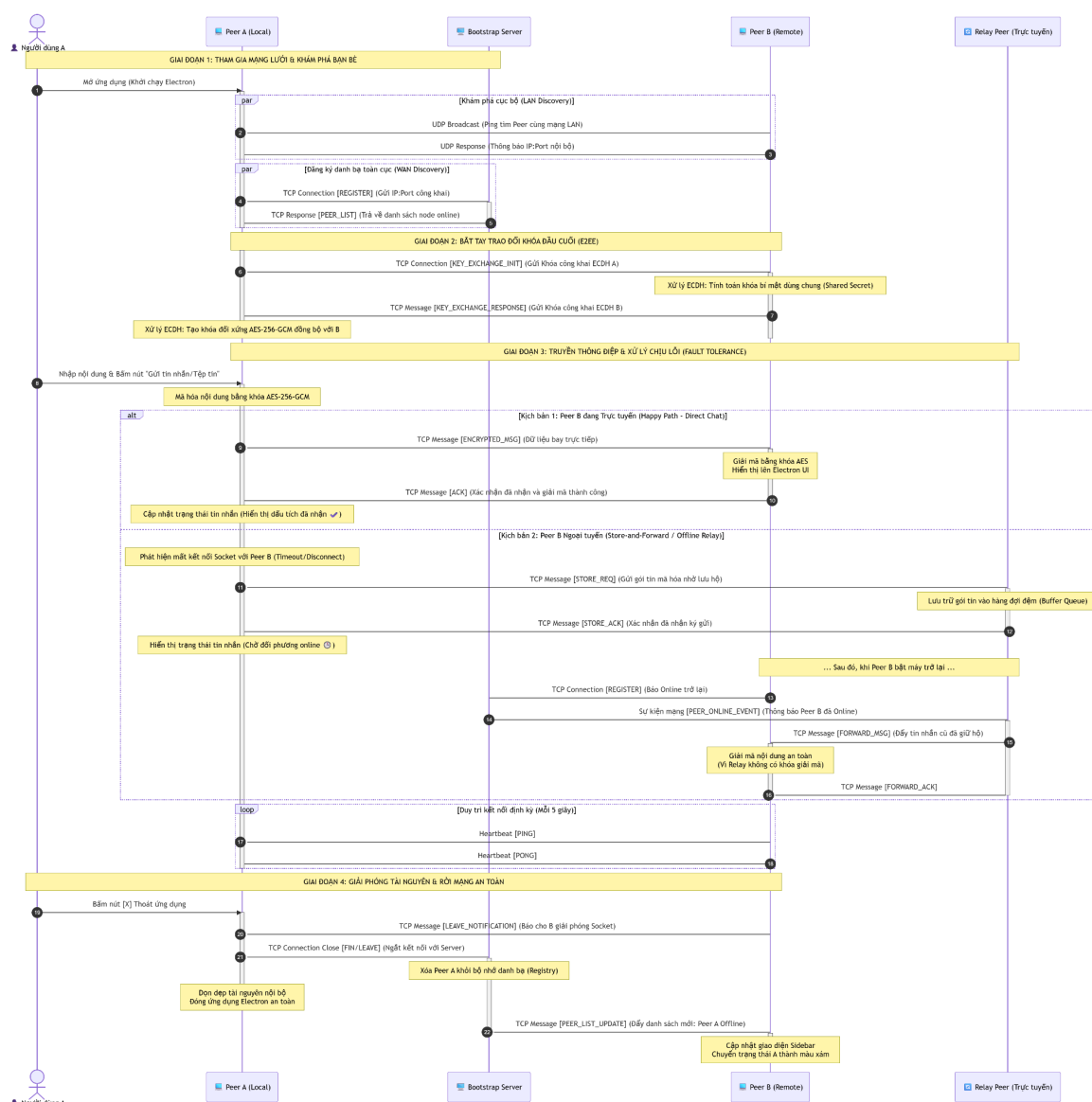
Nhờ sự kết hợp chặt chẽ giữa Heartbeat và Store-and-Forward, hệ thống kiến tạo nên một mạng lưới P2P cực kỳ bền bỉ, có khả năng tự động thích nghi và phục hồi trước mọi biến động về mặt vật lý của hạ tầng mạng.

Lưu ý: Toàn bộ các hằng số thời gian và ngưỡng giới hạn (`HEARTBEAT_INTERVAL`, `HEARTBEAT_TIMEOUT`, `ACK_TIMEOUT`, `MAX_RETRIES`) được định nghĩa tập trung trong tệp `src/config/constants.js`. Nhà quản trị hệ thống có thể điều chỉnh các giá trị này để phù hợp với đặc điểm mạng (LAN ổn định hay WAN biến động cao) mà không cần can thiệp vào mã nguồn lõi.

4.5. Tổng kết luồng hoạt động vòng đời hệ thống (Lifecycle)

Sau khi đi sâu phân tích chi tiết từng cơ chế cốt lõi (Kiến trúc phân lớp, Giao thức, Khám phá mạng và Chịu lỗi), mục này sẽ tích hợp tất cả các mảnh ghép lại để trình bày một bức tranh toàn cảnh về Vòng đời của một Nút mạng (Peer Lifecycle) từ lúc khởi động cho đến khi ngắt kết nối.

Sơ đồ tuần tự tổng quát dưới đây mô tả sự tương tác xuyên suốt giữa các tác nhân: Người dùng (UI), Máy khách (Local Peer), Máy chủ trung tâm (Bootstrap Server), Máy đích (Remote Peer) và Trạm giữ hộ (Relay Peer).



Hình 4.5. Sơ đồ Tuần tự Tổng quát (Master Sequence Diagram)

Toàn bộ vòng đời hoạt động của hệ thống được chia thành 4 giai đoạn logic khép kín:

1. Giai đoạn 1: Tham gia mạng lưới & Khám phá bạn bè (Network Join & Discovery)

- Ngay khi người dùng khởi chạy ứng dụng (Electron App), Lóp Lỗi (Peer) đồng thời kích hoạt hai tiến trình song song (Khởi par):
 - + Dò tìm nội bộ (LAN): Bắn sóng vô tuyến UDP Broadcast để tìm kiếm các đồng nghiệp trong cùng mạng nội bộ.
 - + Đăng ký toàn cầu (WAN): Thiết lập kết nối TCP lên Bootstrap Server để ghi danh (Gửi gói REGISTER) và tải về bản đồ mạng lưới hiện tại (Nhận gói PEER_LIST).
- Kết quả: Danh bạ định tuyến (knownPeers) được thiết lập hoàn chỉnh, sẵn sàng cho việc giao tiếp.

2. Giai đoạn 2: Bắt tay Trao đổi khóa đầu cuối (E2EE Handshake)

- Khi người dùng kích hoạt hành động "Gửi tin nhắn" tới một Peer B (đang trực tuyến) lần đầu tiên, hệ thống sẽ tạm giữ tin nhắn lại và ưu tiên khởi chạy tiến trình thỏa thuận khóa.
- Máy A gửi Public Key (KEY_EXCHANGE_INIT), Máy B đáp trả bằng Public Key của nó (KEY_EXCHANGE_RESPONSE).
- Kết quả: Cả hai máy đều độc lập tính toán ra Khóa bí mật chung (Shared Secret), dẫn xuất ra khóa phiên AES-256-GCM. Từ thời điểm này, kênh truyền đã được bảo mật 100%.

3. Giai đoạn 3: Truyền thông điệp & Xử lý chịu lỗi (Messaging & Fault Tolerance)

- Trong giai đoạn này, luồng thực thi có thể rẽ nhánh (Khởi alt):
 - + Kịch bản Happy Path (Trực tiếp): Tin nhắn được mã hóa E2EE và bắn thẳng qua TCP. Máy nhận giải mã, hiển thị lên UI và trả về tín hiệu ACK. Máy gửi hiển thị "Tích V (Đã nhận)".
 - + Kịch bản Ngoại tuyến (Store-and-Forward): Nếu Máy B rớt mạng (Socket Timeout), bản tin mã hóa lập tức được chuyển hướng sang nhờ một Trạm lân cận (Relay Peer) giữ hộ. Khi Máy B online trở

lại (nhận biết qua sự kiện mạng), Trạm giữ hộ sẽ tự động chuyển tiếp (Forward) bản tin này về đích một cách an toàn.

- Xuyên suốt giai đoạn này, cứ mỗi 3 giây, cơ chế Heartbeat (PING/PONG) lại được kích hoạt nhằm để duy trì tuổi thọ cho các Socket đang mở.

4. Giai đoạn 4: Giải phóng tài nguyên & Rời mạng an toàn (Graceful Teardown)

- Khi người dùng bấm nút [X] hoặc [DISCONNECT] để thoát ứng dụng, hệ thống không đóng đột ngột. Thay vào đó, nó thực hiện quy trình rời mạng an toàn:
 - + Gửi thông báo LEAVE tới tất cả các Peer đang kết nối để họ chủ động giải phóng Socket.
 - + Máy chủ lập tức phát sóng cập nhật (PEER_LIST_UPDATE) cho toàn mạng biết rằng Máy A đã ngoại tuyến, giúp giao diện người dùng (Sidebar) của các máy khác chuyển trạng thái của A thành màu xám.
- Kết quả: Ứng dụng dọn dẹp sạch sẽ tài nguyên phần cứng nội bộ, đóng các cổng lắng nghe và thoát an toàn.

CHƯƠNG 5: CÀI ĐẶT - KIỂM THỬ

5.1. Triển khai và Cài đặt

5.1.1. Tổ chức cấu trúc mã nguồn

Dự án được phân chia thành hai thành phần chính: Ứng dụng Client (Desktop) và Máy chủ Bootstrap. Cấu trúc mã nguồn được tổ chức chặt chẽ theo nguyên lý Clean Architecture, phân tách rõ ràng giao diện người dùng (UI) và logic điều phối mạng (Core Logic).

```
Code/
├── bootstrap-server/
│   └── server.js          # Máy chủ duy trì Registry tập trung (Node.js)
├── client/
│   ├── main.js           # Điểm neo vòng đời Electron (Main Process)
│   ├── cli.js            # Giao diện Command Line (Dùng cho test tự động)
│   └── gui/              # Giao diện đồ họa (Renderer Process)
│       ├── styles.css
│       ├── index.html
│       └── renderer.js
├── src/                  # Thư mục mã nguồn Lỗi P2P
│   ├── core /Peer.js     # Lỗi điều phối (Facade pattern)
│   ├── network/          # Lớp Giao tiếp & Khám phá mạng
│   │   ├── UDPHandler.js
│   │   └── TCPHandler.js
│   ├── discovery/
│   │   └── BootstrapClient.js # Client giao tiếp với Bootstrap Server
│   ├── chat/             # Lớp Nghiệp vụ tin nhắn
│   │   ├── GroupChat.js
│   │   ├── GlobalChat.js
│   │   ├── DirectChat.js
│   │   └── MessageQueue.js
│   ├── security/         # Lớp Mật mã & Bảo mật E2EE
│   │   ├── Crypto.js
│   │   └── KeyExchange.js
├── tests/
│   └── churn-test.js      # Script mô phỏng biến động mạng lưới
```

5.1.2. Hiện thực hóa các Module cốt lõi

Để minh chứng cho việc chuyển hóa thiết kế thành mã nguồn, dưới đây trích dẫn hai thuật toán trọng yếu nhất của hệ thống:

a. Trích đoạn mã hóa và dẫn xuất khóa (AES-256-GCM & SHA-256):

Lớp Crypto không sử dụng trực tiếp sharedSecret sinh ra từ thuật toán ECDH để làm khóa mã hóa, mà tuân thủ chuẩn KDF (Key Derivation Function) bằng cách băm qua SHA-256 để dẫn xuất ra khóa phiên (Session Key) 256-bit chuẩn xác.

```
// Trích xuất từ: src/security/Crypto.js
static encrypt(plainText, sharedSecret) {
  // 1. Dẫn xuất khóa phiên chuẩn 32-byte (256-bit)
  const key = this.deriveKey(sharedSecret);
  const iv = crypto.randomBytes(12); // IV chuẩn cho chế độ GCM
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);

  // 2. Mã hóa dữ liệu bản rõ
  let encrypted = cipher.update(plainText, 'utf8', 'hex');
  encrypted += cipher.final('hex');

  // 3. Trả về cấu trúc an toàn không chứa khóa giải mã
  return {
    iv: iv.toString('hex'),
    content: encrypted,
    tag: cipher.getAuthTag().toString('hex')
  };
}
```

b. Trích đoạn Cơ chế Đảm bảo truyền tải (Message Queue & Retry):

Hệ thống mạng P2P luôn tiềm ẩn rủi ro rớt mạng. Lớp MessageQueue triển khai một vòng lặp đệ quy bất đồng bộ, sử dụng cơ chế Timer để tự động gửi lại (Retry) nếu chưa nhận được tín hiệu ACK từ máy đích.

```
// Trích xuất từ: src/network/MessageQueue.js
_doSend(seq) {
  const item = this.pending.get(seq);
  if (!item) return;

  if (!this.peer.sendToPeer(item.target, item.message)) {
    // Bắt lỗi kết nối vật lý ngay lập tức
    this.pending.delete(seq);
    item.reject(new Error(Không thể kết nối tới
```

```

    ${item.target}));
    return;
  }

  // Hẹn giờ kiểm tra ACK (ACK_TIMEOUT = 5000ms)
  item.timer = setTimeout(() => {
    if (item.retries < MAX_RETRIES) {
      item.retries++;
      logger.warn(Đang thử gửi lại gói tin #${seq}...);
      this._doSend(seq);
    } else {
      this.pending.delete(seq);
    }
  }, ACK_TIMEOUT);
  item.reject(new Error(Gửi tin nhắn #${seq} thất bại
sau ${MAX_RETRIES} lần thử.));
}
}

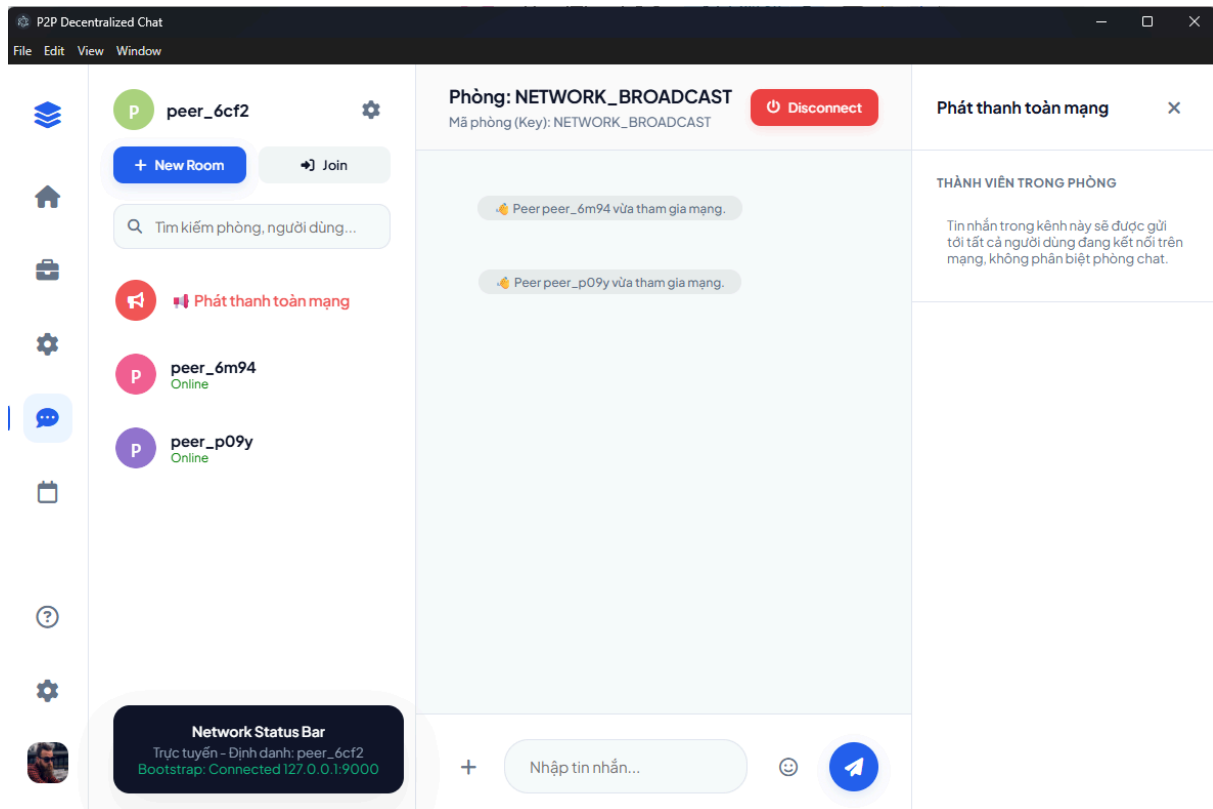
```

Chú thích về Cơ chế Store-and-Forward: Lời gọi `item.reject()` ở trên không làm treo (crash) ứng dụng. Lỗi này sẽ được Catch (bắt lại) ở lớp Peer. Nhờ tuân thủ nguyên lý Single Responsibility, MessageQueue chỉ làm nhiệm vụ đếm Retry; việc kích hoạt luồng Ký gửi ngoại tuyến (Store-and-Forward) để tìm Relay Node dự phòng sẽ do lớp Peer tự động quyết định.

5.1.3. Giao diện Người dùng (GUI)

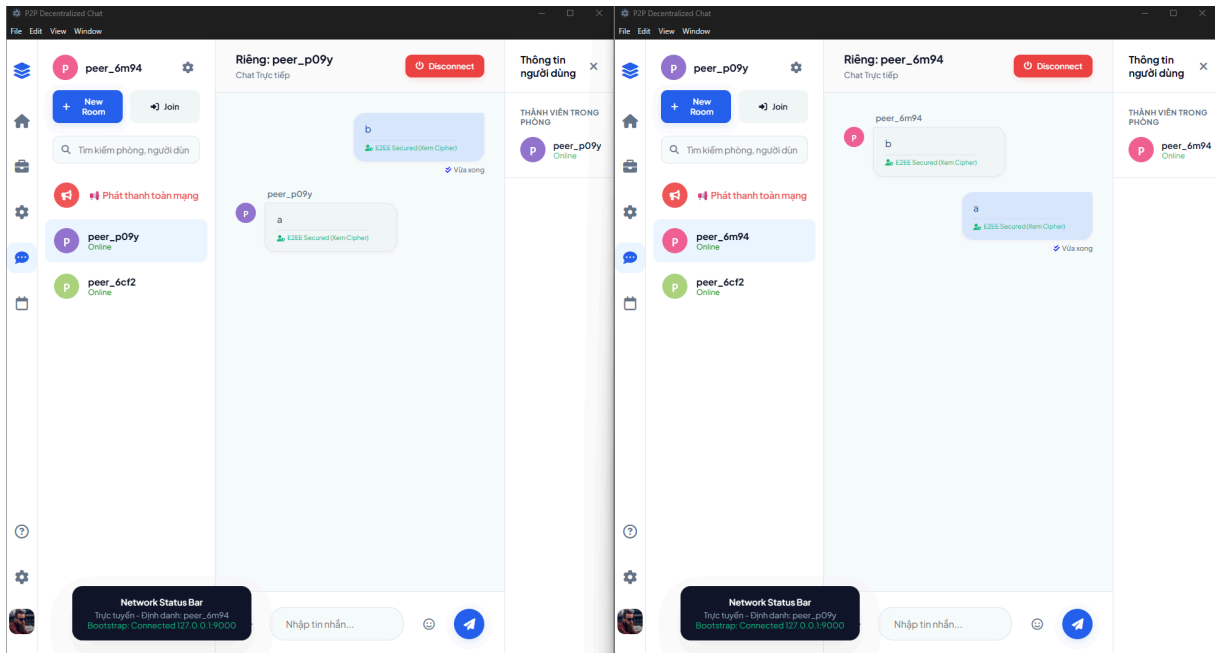
a. Giao diện người dùng (Hình 5.1) được thiết kế theo phong cách tối giản (Minimalism) với ba khu vực chức năng chính:

1. Thanh điều hướng (Sidebar): Quản lý định danh cá nhân, hiển thị danh sách các Peer đang trực tuyến trong mạng lưới (cập nhật theo thời gian thực) và các phòng chat (Rooms).
2. Khu vực tương tác (Main Chat): Hiển thị nội dung trao đổi dạng bong bóng (bubble chat). Hệ thống thông báo tự động (Log) các sự kiện mạng như: thiết bị tham gia hay rời khỏi phòng.
3. Thanh trạng thái (Network Status Bar): Nằm ở góc dưới bên trái, đóng vai trò sống còn trong việc hiển thị tình trạng kết nối TCP và trạng thái đồng bộ với Bootstrap Server.



Hình 5.1. Giao diện người dùng ứng dụng P2P Chat

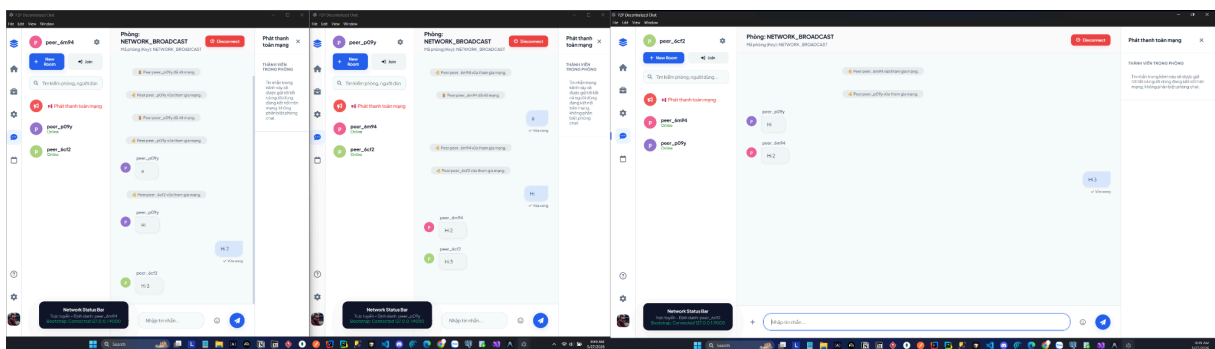
b. Kiểm thử chat 1-1 (Hình 5.2): hai định danh peer_6m94 và peer_p09y đã thiết lập thành công đường hầm bảo mật. Giao diện thể hiện rõ ràng luồng thông tin hai chiều với độ trễ gần như bằng không. Điểm nhấn kỹ thuật nằm ở nhãn 'E2EE Secured' đính kèm dưới mỗi bong bóng chat, kết hợp cùng dấu tích xanh (Vừa xong), khẳng định thông điệp đã đi qua lớp Crypto.js, mã hóa thành công và đã nhận được phản hồi ACK từ máy đích ở tầng TCP



Hình 5.2. Thử nghiệm truyền thông điệp mã hóa trực tiếp (Direct Chat) giữa hai Node.

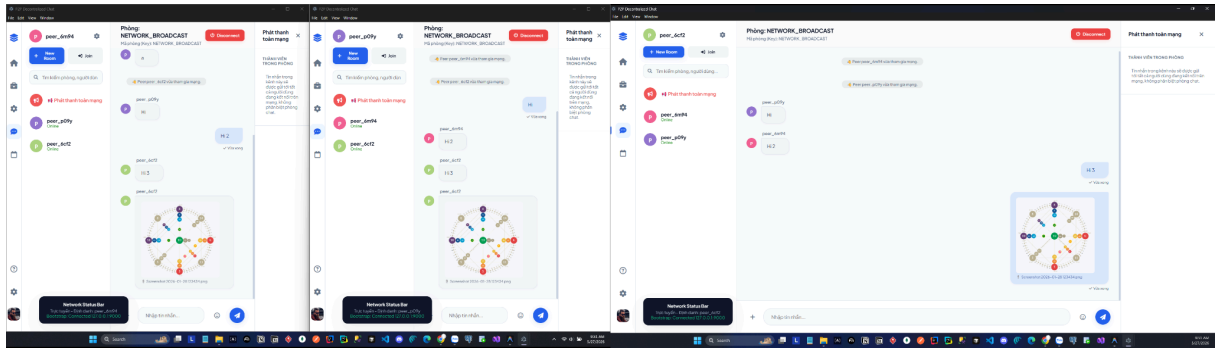
c. Broadcast toàn mạng: hệ thống cung cấp kênh 'Phát thanh toàn mạng' (NETWORK_BROADCAST). Hình 5.3 thể hiện kịch bản kiểm thử khi 3 thiết bị (peer_6m94, peer_p09y, và peer_6cf2) cùng tham gia vào kênh này.

Khi một định danh (ví dụ peer_p09y gửi tin nhắn 'Hi'), Lớp Mạng (Network Layer) không thiết lập một kênh TCP đơn lẻ mà sử dụng thuật toán lặp (Iterative Routing) để nhân bản và phân phối gói tin tới tất cả các Socket đang duy trì kết nối trong danh bạ. Giao diện thể hiện rõ sự đồng bộ thời gian thực (Real-time synchronization): tin nhắn từ bất kỳ Node nào cũng lập tức được hiển thị nhất quán trên màn hình của toàn bộ các Node còn lại trong mạng, chứng minh cơ chế Broadcast hoạt động ổn định mà không xảy ra tình trạng mất gói tin (Packet loss) hay kẹt nghẽn hàng đợi (Queue congestion)."



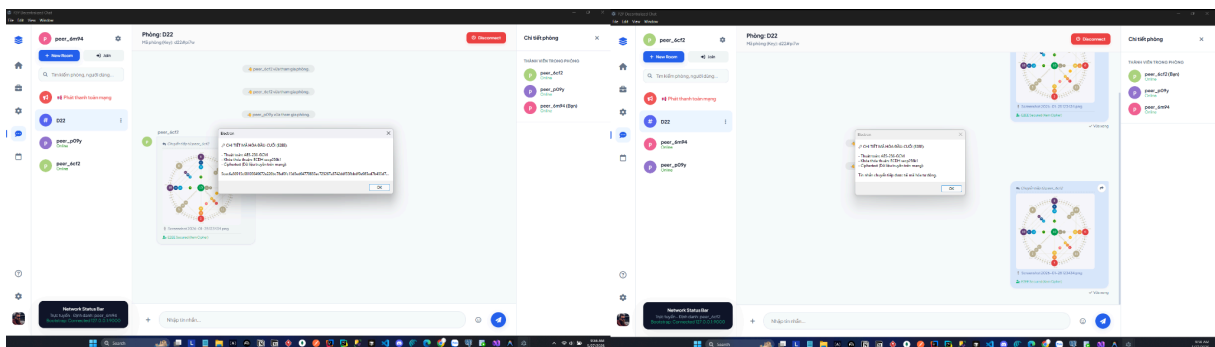
Hình 5.3. Giao diện thử nghiệm Broadcast message toàn mạng

d. Truyền tệp tin đa phương tiện (Hình 5.4). Người dùng peer_p09y đính kèm một tệp tin hình ảnh. Lớp lõi lập tức băm tệp tin này thành chuỗi Base64 và mã hóa trước khi đẩy qua Socket. Ở đầu nhận, peer_6m94 và peer_6cf2 tự động giải mã AES-256-GCM và kết xuất (render) luồng byte đó ngược trở lại thành hình ảnh trực quan trên DOM (Document Object Model) mà không xảy ra hiện tượng tràn bộ nhớ đệm hay nghẽn luồng xử lý UI.



Hình 5.4. Kịch bản truyền dữ liệu đa phương tiện (Hình ảnh) qua kênh mã hóa E2EE.

e. Xem chi tiết mã hóa (Ciphertext) (Hình 5.5), hệ thống tích hợp chức năng 'Xem Cipher'. Khi người dùng nhấp vào chứng chỉ bảo mật của một tin nhắn bất kỳ, hệ thống sẽ trích xuất và hiển thị dữ liệu gốc (Raw Data) đang lưu chuyển trên đường truyền vật lý. Hộp thoại xác nhận rõ thông điệp đã được bảo vệ bởi thuật toán AES-256-GCM dựa trên khóa phiên sinh ra từ ECDH secp256k1, đồng thời hiển thị Ciphertext (Chuỗi nhị phân mã hóa vô nghĩa) để chứng minh rằng ngay cả khi có kẻ bắt trộm gói tin ở tầng mạng, việc đọc trộm nội dung là bất khả thi.

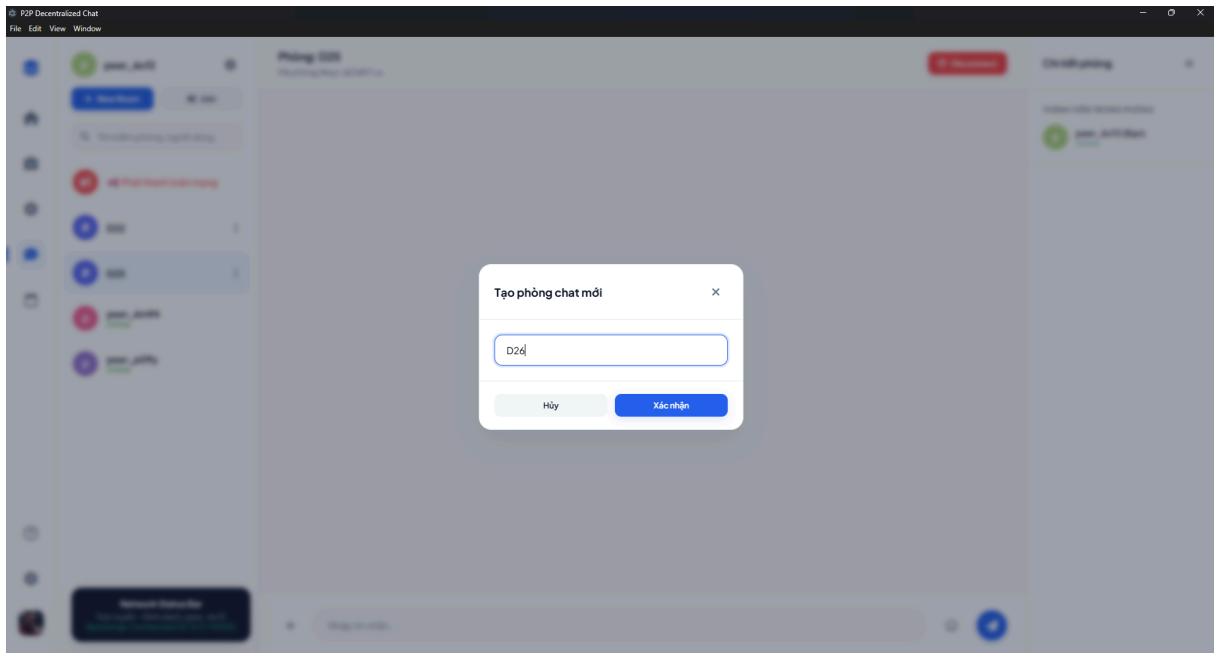


Hình 5.5. Hộp thoại trích xuất cấu trúc dữ liệu mật mã thực tế của gói tin.

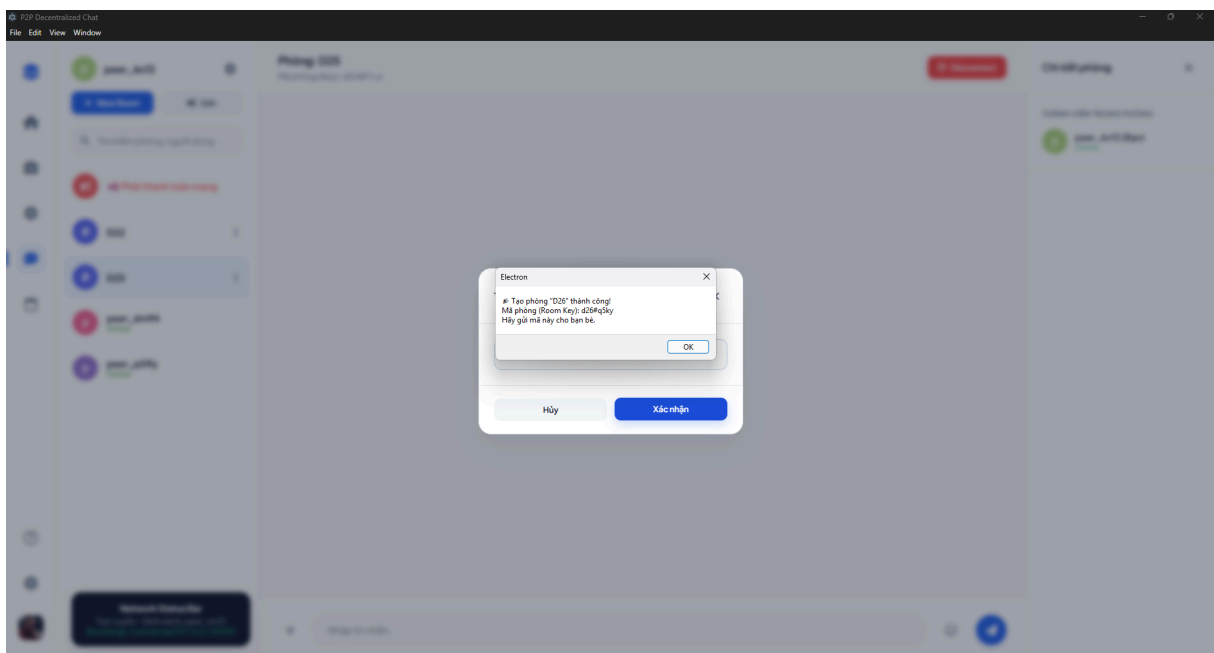
f. Giao diện Tạo phòng (Create Room)

Tạo phòng nhóm (Hình 5.6, 5.7, 5.8): Người dùng nhấn nút “+ New Room”, nhập tên phòng, hệ thống sinh ra một mã phòng (Room Key) duy nhất (ví dụ

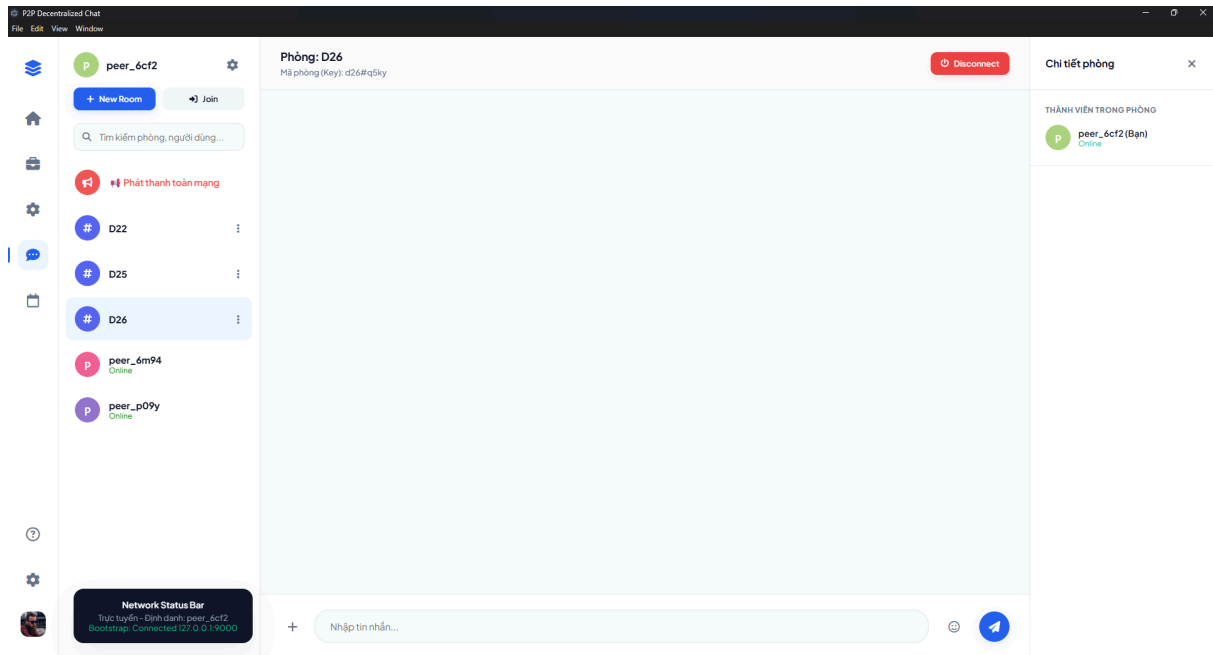
d26#q5ky). Mã phòng này đóng vai trò định danh logic, giúp cô lập luồng thông điệp của nhóm khỏi mạng lưới chung. Giao diện cho phép sao chép nhanh mã phòng để chia sẻ ngoài băng tần (out-of-band).



Hình 5.6. Giao diện Tạo phòng

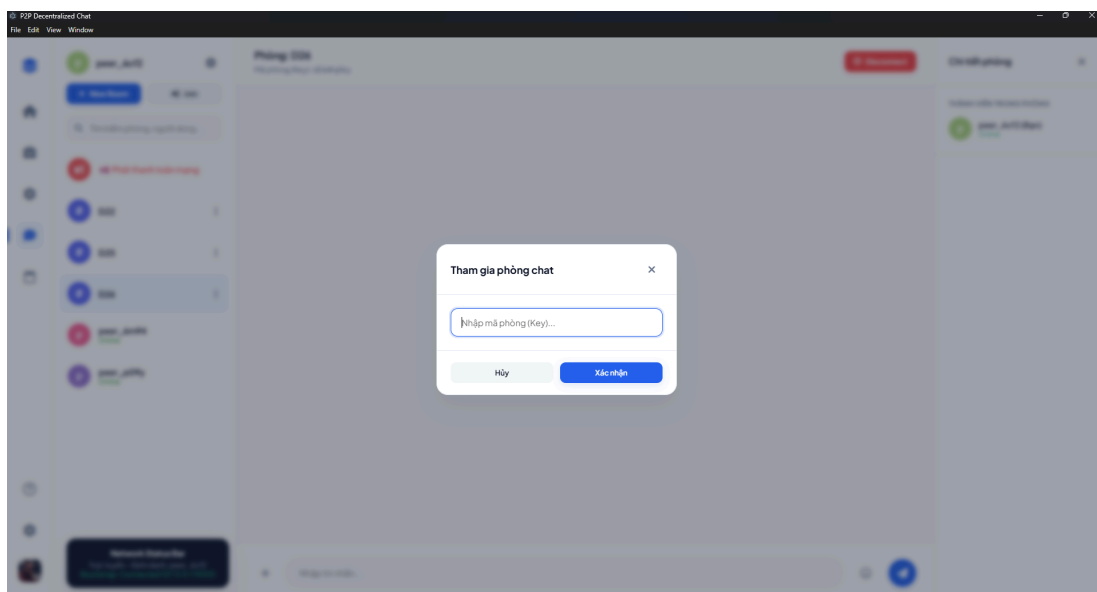


Hình 5.7. Giao diện khi Tạo phòng thành công

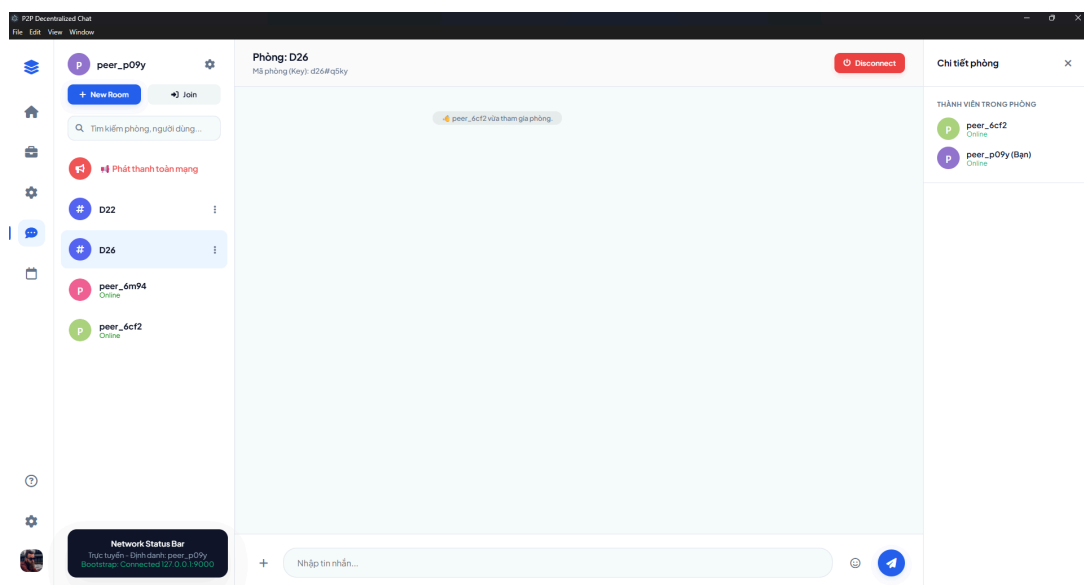


Hình 5.8. Giao diện vào phòng D26 khi mới tạo

g. Giao diện Gia nhập phòng (Join Room): Tham gia phòng nhóm (Hình 5.9, 5.10): Người dùng nhấn “Join Room” và nhập mã phòng hợp lệ. Hệ thống không kết nối đến máy chủ trung tâm để tải lịch sử; thay vào đó, Lớp Giao tiếp mạng cập nhật bảng định danh cục bộ, gắn thẻ thiết bị vào danh sách thành viên của Room Key tương ứng. Từ thời điểm này, mọi gói tin JSON gửi vào phòng đều mang thêm trường roomId, các Node nhận được sẽ kiểm tra và chỉ hiển thị nếu khớp, đảm bảo tính riêng tư của luồng tin nhắn nhóm



Hình 5.9. Giao diện Gia nhập phòng (Join)



Hình 5.10. Giao diện sau khi vào thành công phòng D26

5.2. Kịch bản và Kết quả Kiểm thử (Testing & Evaluation)

5.2.1. Các kịch bản kiểm thử tính năng (Functional Test)

Mã	Tên	Mục tiêu	Các bước thực hiện	Kết quả mong đợi
01	Khởi tạo và Đăng ký vào mạng	Các thiết bị người dùng (Peer) kết nối và đăng ký thành công lên máy chủ danh bạ trung tâm (Bootstrap Server).	1. Khởi động máy chủ trung tâm. 2. Khởi động ứng dụng người dùng và cho kết nối vào máy chủ.	Máy chủ ghi nhận thiết bị mới đăng ký. Thiết bị nhận được thông báo đã gia nhập mạng lưới thành công.
02	Tự động khám phá và Kết nối trực tiếp	Thiết bị tự nhận diện được những người dùng khác và thiết lập mạng kết nối trực tiếp.	Cho nhiều thiết bị cùng kết nối vào máy chủ trung tâm (hoặc cùng chung một mạng WiFi/LAN).	Các thiết bị tự động tải về danh bạ, hiển thị tên những người đang trực tuyến và tự động thiết lập kết nối dữ liệu trực tiếp (TCP P2P) với nhau.
03	Sinh và Trao đổi khóa bảo mật	Hệ thống tự động sinh bộ khóa mật mã và thiết lập kênh giao tiếp an toàn giữa hai người dùng.	1. Cho hai thiết bị kết nối với nhau lần đầu tiên. 2. Kiểm tra tiến trình bắt tay bảo mật ở nền tảng.	Hai thiết bị tự động trao đổi khóa, tính toán thành công ra cùng một mã khóa bí mật chung (Shared Secret) và hiển thị chung một chuỗi đối chiếu bảo mật giống hệt nhau.
04	Nhắn tin trực tiếp (Mã hóa an toàn)	Tin nhắn gửi đi được mã hóa bảo mật và chỉ có thể đọc được ở phía người nhận gốc.	1. Gửi một tin nhắn bất kỳ từ thiết bị này sang thiết bị khác. 2. Theo dõi trạng thái dữ liệu được đẩy ra môi trường mạng.	Nội dung trên đường truyền bị mã hóa hoàn toàn thành các ký tự vô nghĩa (Ciphertext). Máy người nhận bắt được gói tin, tự giải mã và hiển thị lại nội dung gốc lên màn hình.

05	Cập nhật trạng thái tin nhắn (Xác nhận gửi)	Đảm bảo hệ thống báo hiệu cho người gửi biết tin nhắn đã đến đích.	Gửi tin nhắn thành công tới một người đang trực tuyến.	Ngay khi nhận được thư, máy người nhận ngầm gửi lại một tín hiệu xác nhận. Giao diện máy người gửi tự động cập nhật biểu tượng trạng thái tin nhắn thành "Đã gửi/Đã nhận".
06	Nhắn tin đa luồng (Nhóm và Toàn mạng)	Tính năng trò chuyện nhiều người trong một phòng (Room) hoặc phát thông báo chung.	1. Tạo một phòng chat và gửi tin nhắn vào phòng có nhiều thành viên. 2. Gửi một thông báo lên kênh Chat Toàn Cầu.	- Trong phòng chat: Tin nhắn được mã hóa riêng biệt cho từng thành viên và hiển thị chính xác trong giao diện nhóm. - Toàn mạng: Mọi thiết bị đang kết nối đều nhận được tin nhắn dạng thông báo và hiển thị ở kênh chung.
07	Chuyển tiếp tin nhắn ngoại tuyến (Store-and-Forward)	Hệ thống tự động nhờ các thiết bị xung quanh cất giữ tin nhắn khi người nhận đang tắt máy, và giao lại khi họ bật máy.	1. Tắt kết nối thiết bị của người nhận (Người B). 2. Người gửi (Người A) gửi một tin nhắn cho Người B. 3. Sau đó, bật lại thiết bị của Người B.	- Lúc gửi: Hệ thống của Người A tự động đóng gói tin nhắn nhờ vả và gửi sang các thiết bị trung gian khác đang trực tuyến. Thiết bị trung gian thông báo đã lưu giữ hộ. - Lúc bật lại: Thiết bị trung gian phát hiện Người B vừa trực tuyến, tự động xả và chuyển giao toàn bộ tin nhắn lưu hộ. Người B nhận được thư và tự động giải mã thành công.

08	Tự động dọn dẹp kết nối rớt mạng (Heartbeat Timeout)	Phát hiện và loại bỏ các thiết bị ngắt kết nối đột ngột (đứt cáp, sập nguồn) khỏi mạng lưới để giải phóng tài nguyên.	1. Ngắt mạng đột ngột của một thiết bị đang trực tuyến. 2. Chờ các thiết bị khác thực hiện vòng kiểm tra sức khỏe định kỳ (Heartbeat).	Các thiết bị còn lại tự động phát hiện đối phương không phản hồi. Tự động ngắt kết nối an toàn, dọn dẹp bộ nhớ và xóa người dùng đó khỏi danh sách trực tuyến.
09	Mô phỏng biến động mạng (Network Churn Simulation)	Kiểm tra độ ổn định của hệ thống khi peer liên tục tham gia và rời mạng (churn), đánh giá cơ chế heartbeat và giải phóng tài nguyên.	1. Dùng script start-churn.ps1 2. Mỗi peer có vòng đời 10 giây: đăng ký, kết nối P2P, gửi tin nhắn, rồi tự ngắt. 3. Theo dõi log terminal, kiểm tra phát hiện peer rời mạng qua heartbeat (PING 3s, timeout 10s).	- Tất cả peer ảo đều hoàn thành vòng đời mà không bị crash. - Tỷ lệ lỗi = 0%. - Heartbeat phát hiện chính xác peer rời mạng trong ngưỡng timeout đã cấu hình (3 giây PING, timeout 10 giây). - Không xảy ra rò rỉ socket hay bộ nhớ sau khi kết thúc mô phỏng. - Cơ chế mã hóa ECDH/AES-GCM vẫn thiết lập thành công 100% trong môi trường biến động.

Bảng 5.1. Kịch bản kiểm thử tính năng

5.2.2. Kiểm thử Mô phỏng biến động mạng (Network Churn Simulation)

Mạng P2P thực tế luôn đối mặt với rủi ro các thiết bị tắt/mở liên tục. Kịch bản (Mô phỏng Churn) được xây dựng bằng công cụ tự động hóa churn-test.js để đo lường giới hạn chịu đựng (Stress Test) và khả năng dọn dẹp tài nguyên (Garbage Collection) thông qua cơ chế Heartbeat PING/PONG.

Quy trình chạy:

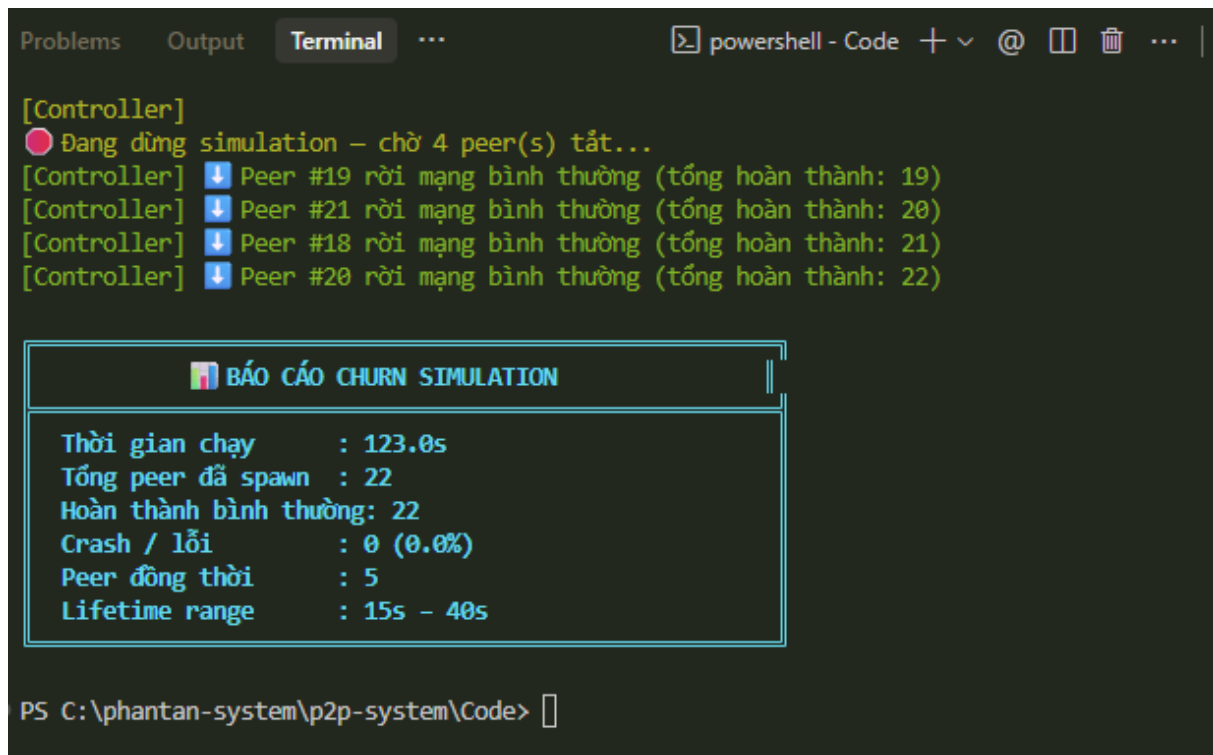
Powershell

```
powershell -ExecutionPolicy Bypass -File .\start-churn.ps1
```

Hệ thống tự động sinh ra các Peer ảo (Lifetime từ 15-40 giây) liên tục kết nối, gửi tin nhắn và ngẫu nhiên sập nguồn để tạo môi trường đứt gãy kết nối liên tục.

Hạng mục đo lường	Số liệu đo đạc thực tế	Đánh giá & Kết luận
Thời gian chạy mô phỏng	123.0s	Chạy liên tục dưới áp lực cao
Tổng số Peer ảo đã sinh	22	Mạng lưới giả lập quy mô vừa
Số Peer hoàn thành chu kỳ	22/22	Không xảy ra hiện tượng kẹt tài nguyên
Số Peer treo/ Crash do lỗi	0	Ứng dụng cực kỳ ổn định. Quản lý luồng tốt
Lọc Node chết (Heartbeat)	PING: 3s/ Timeout: 10s	Phát hiện và đóng Socket chính xác
Mã hóa ECDH/ AES-GCM	Thiết lập khóa 100% thành công	Vượt rào cản bất đồng bộ hoàn hảo

Bảng 5.2. Thống kê kết quả thực nghiệm tự động (Automated Test Metrics)



```
[Controller]
Đang dừng simulation – chờ 4 peer(s) tắt...
[Controller] Peer #19 rời mạng bình thường (tổng hoàn thành: 19)
[Controller] Peer #21 rời mạng bình thường (tổng hoàn thành: 20)
[Controller] Peer #18 rời mạng bình thường (tổng hoàn thành: 21)
[Controller] Peer #20 rời mạng bình thường (tổng hoàn thành: 22)
```

BÁO CÁO CHURN SIMULATION

Thời gian chạy	: 123.0s
Tổng peer đã spawn	: 22
Hoàn thành bình thường:	22
Crash / lỗi	: 0 (0.0%)
Peer đồng thời	: 5
Lifetime range	: 15s – 40s

```
PS C:\phantan-system\p2p-system\Code>
```

Hình 5.11 chụp kết quả log Terminal - BÁO CÁO CHURN SIMULATION.

Kết luận thực nghiệm: Hệ thống đã vượt qua bài kiểm tra chịu tải với tỉ lệ lỗi 0.0%. Cơ chế quản lý vòng đời bằng Timeout 10s kết hợp tự động gửi lại (Retry) chứng minh hệ thống có năng lực Tự phục hồi (Self-healing) cực kỳ mạnh mẽ trước các bất ổn vật lý của hạ tầng mạng ngang hàng.

CHƯƠNG 6: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

6.1. Tổng kết kết quả đạt được

Đồ án đã hoàn thành xuất sắc mục tiêu xây dựng một hệ thống chat ngang hàng (P2P) hybrid có bảo mật, đáp ứng toàn bộ các yêu cầu chức năng cơ bản và các yêu cầu nâng cao đã đề ra. Những kết quả kỹ thuật nổi bật có thể kể đến:

Về kiến trúc hệ thống:

- Xây dựng thành công kiến trúc Hybrid P2P ba tầng: Tầng Internet/WAN (Bootstrap Server), Tầng P2P/Mesh Network (các peer và kết nối trực tiếp), và Tầng LAN Discovery (UDP Broadcast). Mô hình “Servent” (Server + Client) trên mỗi peer cho phép hệ thống vừa có khả năng khởi tạo dễ dàng qua máy chủ trung tâm, vừa giữ được bản chất phân tán và tự trị.
- Áp dụng kiến trúc phân lớp (Layered Architecture) kết hợp với mẫu Facade: tách bạch rõ ràng giữa giao diện (Electron), lớp lõi điều phối (Peer), lớp bảo mật (KeyExchange, Crypto) và lớp mạng (TCP, UDP, BootstrapClient). Nhờ đó, hệ thống có tính module cao, dễ bảo trì và mở rộng.

Về giao thức trao đổi thông điệp:

- Thiết kế giao thức JSON-over-TCP với ngắt dòng (newline-delimited), giải quyết triệt để vấn đề dính gói tin (sticky packets) trên luồng TCP.
- Định nghĩa bộ mã thông điệp phong phú gồm hơn 15 loại (REGISTER, PEER_LIST, DIRECT_CHAT, GROUP_CHAT, GLOBAL_CHAT, ACK, STORE_AND_FORWARD, PING/PONG, LEAVE...), phục vụ đầy đủ cho cả mặt phẳng điều khiển và mặt phẳng dữ liệu.
- Tích hợp mã hóa đầu-cuối (E2EE) trực tiếp trong cấu trúc payload: chỉ mã hóa nội dung (content) bằng AES-256-GCM, giữ nguyên các trường định tuyến (type, from, to, roomId) để các trạm trung gian có thể xử lý mà không cần giải mã. Khóa phiên được thỏa thuận an toàn qua ECDH (prime256v1) và dẫn xuất bằng SHA-256, đảm bảo tính bí mật và toàn vẹn tuyệt đối cho dữ liệu người dùng.

Về cơ chế khám phá peer:

- Triển khai thành công ba cơ chế khám phá song song: UDP Broadcast cho mạng LAN (cho phép chat nội bộ ngay cả khi mất Internet), Bootstrap Server Sync cho mạng WAN (đồng bộ danh sách peer toàn cục), và Implicit TCP Handshake (học peerId từ message đầu tiên, tối ưu băng thông và tăng tốc độ thiết lập mạng lưới). Nhờ vậy, hệ thống đạt được tính sẵn sàng cao trong mọi điều kiện hạ tầng mạng.

Về xử lý lỗi và đảm bảo độ tin cậy:

- Cài đặt cơ chế Heartbeat (PING/PONG) với tần suất 3 giây và ngưỡng timeout 10 giây, kết hợp Stale Peer Detection để phát hiện nhanh các peer mất kết nối đột ngột, giải phóng tài nguyên kịp thời.
- Xây dựng cơ chế Message Queue với ACK và Retry: mỗi tin nhắn được gán số thứ tự (seq), tự động gửi lại tối đa 3 lần nếu không nhận được ACK sau 5 giây, đảm bảo độ tin cậy trên kênh truyền không ổn định.
- Phát triển cơ chế Store-and-Forward phân tán: khi peer đích ngoại tuyến, tin nhắn (đã mã hóa) được các peer trung gian lưu tạm trên RAM và tự động chuyển tiếp ngay khi đích trực tuyến trở lại. Cơ chế này hoạt động hoàn toàn tự động, không làm mất dữ liệu, đồng thời Relay Peer không bao giờ có khóa giải mã, đảm bảo tính riêng tư tuyệt đối.
- Cung cấp công cụ churn-test.js để mô phỏng môi trường biến động cao (22 peer ảo, tỉ lệ rời mạng 10%/phút), qua đó kiểm chứng khả năng chịu lỗi và tự phục hồi của hệ thống.

Về giao diện và triển khai:

- Xây dựng ứng dụng desktop hoàn chỉnh bằng Electron, giao diện trực quan, hỗ trợ đầy đủ các thao tác: xem danh sách peer, chat riêng, chat nhóm, broadcast toàn mạng, gửi file (Base64), hiển thị trạng thái mã hóa.
- Cung cấp các kịch bản triển khai thực tế trên LAN, qua Radmin VPN và Cloud VPS, cùng các script PowerShell tự động hóa, giúp hệ thống sẵn sàng cho việc sử dụng thực tế ngay lập tức.

Về kiểm thử và đánh giá:

- Thực hiện thành công các kịch bản kiểm thử, bao gồm kiểm thử khám phá mạng, E2EE, store-and-forward, churn simulation, truyền file.
- Mô phỏng churn với 22 peer ảo, tỷ lệ lỗi 0,0%, khẳng định khả năng tự phục hồi và ổn định của hệ thống.

Nhìn chung, đề án đã chứng minh được khả năng áp dụng nhuần nhuyễn các nguyên lý của hệ thống phân tán, mật mã học và lập trình mạng vào một sản phẩm hoàn chỉnh, có giá trị sử dụng thực tiễn và giá trị học thuật cao.

6.2. Phương hướng phát triển

a. Hoàn thiện cơ chế chịu lỗi cho Bootstrap Server

- Triển khai đồng bộ dữ liệu giữa nhiều instance Bootstrap Server (qua Redis, etcd hoặc cơ chế gossip) để loại bỏ hoàn toàn điểm yếu tập trung. Kết hợp với cơ chế failover tự động và persistent peer cache, đảm bảo peer mới luôn có thể gia nhập mạng WAN ngay cả khi một số server gặp sự cố.

b. Chuyển đổi sang mô hình lưu trữ bền vững

- Thay thế lưu trữ trong RAM bằng cơ sở dữ liệu nhúng (SQLite, LevelDB) cho hàng đợi Store-and-Forward, danh sách peer, lịch sử tin nhắn và message ID đã xử lý. Điều này giúp dữ liệu không bị mất khi peer khởi động lại hoặc gặp sự cố mất điện.

c. Nâng cấp cơ chế truyền file

- Áp dụng kỹ thuật chia nhỏ (chunking) và truyền song song nhiều chunk, hỗ trợ gửi file dung lượng lớn (hàng trăm MB, GB). Tích hợp nén trước khi mã hóa và gửi dữ liệu nhị phân trực tiếp (thay vì Base64) để giảm tải băng thông. Bổ sung cơ chế tiếp tục truyền khi bị gián đoạn (resumable transfer).

d. Cải thiện khả năng xuyên NAT

- Tích hợp các giao thức STUN/TURN (ví dụ: thư viện node-stun, coturn) để hỗ trợ kết nối trực tiếp giữa các peer sau NAT đối xứng. Đối với các trường hợp không thể kết nối P2P, sử dụng TURN server làm relay trung gian, đảm bảo tỷ lệ kết nối thành công cao trong môi trường mạng thực tế.

e. Xây dựng cơ chế xác thực danh tính

- Áp dụng hạ tầng khóa công khai (PKI) với chứng chỉ số tự ký hoặc được cấp bởi một cơ quan chứng nhận riêng. Nghiên cứu sử dụng định danh phi tập trung (DID) kết hợp với blockchain nhẹ để lưu trữ và xác minh khóa công khai, ngăn chặn giả mạo peer ID.

f. Tối ưu giao thức truyền tin

- Điều chỉnh tham số retry và timeout động dựa trên thời gian phản hồi thực tế (RTT). Gộp nhiều tin nhắn nhỏ vào cùng một gói TCP để giảm overhead. Cân nhắc chuyển từ TCP raw sang WebRTC DataChannel để có cơ chế xuyên NAT tốt hơn và hỗ trợ trình duyệt.

g. Phát triển giao diện người dùng chuyên nghiệp

- Xây dựng lại GUI bằng các framework hiện đại như React hoặc Vue, tích hợp thông báo desktop, tìm kiếm tin nhắn, lưu trữ lịch sử cục bộ (IndexedDB), hỗ trợ theme sáng/tối và cải thiện trải nghiệm người dùng tổng thể.

TÀI LIỆU THAM KHẢO

- [1] R. Schollmeier, “A Definition of Peer-to-Peer Networking for the Classification of Peer-to-Peer Architectures and Applications,” in *Proc. 1st Int. Conf. Peer-to-Peer Comput. (P2P '01)*, Linköping, Sweden, 2001, pp. 101–102. doi: 10.1109/P2P.2001.990434.
- [2] J. Dinger and O. P. Waldhorst, “Decentralized Bootstrapping of P2P Systems,” in Handbook of Peer-to-Peer Networking, X. Shen, H. Yu, J. Buford, and M. Akon, Eds. Boston, MA: Springer, 2010, pp. 579–609. doi: 10.1007/978-0-387-09751-0_21.
- [3] A. S. Tanenbaum and M. Van Steen, Distributed Systems: Principles and Paradigms, 2nd ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2007.
- [4] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, “Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications,” in Proc. 2001 Conf. Appl. Technol. Archit. Protocols Comput. Commun. (SIGCOMM '01), San Diego, CA, 2001, pp. 149–160. doi: 10.1145/383059.383071.
- [5] S. A. Baset and H. Schulzrinne, “An Analysis of the Skype Peer-to-Peer Internet Telephony Protocol,” in Proc. 25th IEEE Int. Conf. Comput. Commun. (INFOCOM 2006), Barcelona, Spain, 2006, pp. 1–11. doi: 10.1109/INFOCOM.2006.312.
- [6] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” National Institute of Standards and Technology (NIST), Special Publication 800-38D, Nov. 2007. doi: 10.6028/NIST.SP.800-38D.
- [7] E. Barker, L. Chen, and R. Davis, “Recommendation for Key-Derivation Methods in Key-Establishment Schemes,” National Institute of Standards and Technology (NIST), Special Publication 800-56C Rev. 2, Aug. 2020. doi: 10.6028/NIST.SP.800-56Cr2.
- [8] D. McGrew, K. Igoe, and M. Salter, “Fundamental Elliptic Curve Cryptography Algorithms,” Internet Engineering Task Force (IETF), RFC 6090, Feb. 2011. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6090>.

[9] Node.js Foundation, “Node.js v18.20.4 Documentation – Crypto Module,” 2024. [Online]. Available: <https://nodejs.org/docs/latest-v18.x/api/crypto.html>.

[10] Node.js Foundation, “Node.js v18.20.4 Documentation – Net Module (TCP),” 2024. [Online]. Available: <https://nodejs.org/docs/latest-v18.x/api/net.html>.

[11] Node.js Foundation, “Node.js v18.20.4 Documentation – Dgram Module (UDP),” 2024. [Online]. Available: <https://nodejs.org/docs/latest-v18.x/api/dgram.html>.

[12] Electron Community, “Electron Documentation – Inter-Process Communication (IPC),” 2024. [Online]. Available: <https://www.electronjs.org/docs/latest/tutorial/ipc>.

[13] S. Loreto and S. P. Romano, Real-Time Communication with WebRTC: Peer-to-Peer in the Browser. Sebastopol, CA: O’Reilly Media, 2014.

[14] Ngô Hồng Sơn, Giáo trình Các hệ thống phân tán. Hà Nội: Học viện Công nghệ Bưu chính Viễn thông, 2024.

PHỤ LỤC

PHỤ LỤC A: HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH HỆ THỐNG

A.1. Chuẩn bị môi trường

- Yêu cầu hệ thống đã cài đặt Node.js (v18+). Mở Terminal tại thư mục gốc của mã nguồn (Code/) và chạy lệnh cài đặt thư viện:

Powershell

```
npm install
```

(Lưu ý: Mọi lệnh PowerShell dưới đây đều cần chạy trong thư mục Code/)

A.2. Kịch bản 1: Chạy thử nghiệm trên 1 máy tính (Local)

- Dùng để giả lập nhiều người dùng trên cùng một máy mà không bị xung đột bộ nhớ Cache.
- Bước 1: Bật Máy chủ danh bạ (Bootstrap Server)

```
powershell -ExecutionPolicy Bypass -File .\start-lan-server.ps1
```

- Bước 2: Mở giao diện Người dùng A & B (Bật từng Terminal mới)

```
//Mở Node A:  
powershell -ExecutionPolicy Bypass -File .\start-lan-gui.ps1  
-BootstrapIp 127.0.0.1 -Profile NodeA  
//Mở Node B:  
powershell -ExecutionPolicy Bypass -File .\start-lan-gui.ps1  
-BootstrapIp 127.0.0.1 -Profile NodeB
```

A.3. Kịch bản 2: Triển khai thực tế trên nhiều máy tính (LAN/VPN)

Khi chạy trên nhiều máy tính khác nhau qua WiFi hoặc mạng ảo (Radmin VPN):

1. Trên Máy chủ: Bật Bootstrap Server và lấy địa chỉ IPv4 (VD: 192.168.1.15).
2. Trên Máy trạm (Clients): Bật ứng dụng và trỏ tham số IP về máy chủ:

```
powershell -ExecutionPolicy Bypass -File .\start-lan-gui.ps1  
-BootstrapIp 192.168.1.15 -Profile NodeClient
```

A.4. Kịch bản 3: Tự động hóa kiểm thử mạng (Churn Test)

Công cụ tự động sinh hàng loạt Peer để kiểm tra khả năng chịu lỗi và dọn dẹp bộ nhớ (Heartbeat Timeout) của kiến trúc mạng:

1. Chạy test mặc định (5 Peer ảo, chạy trong 2 phút):

```
powershell -ExecutionPolicy Bypass -File .\start-churn.ps1
```

2. Chạy tải cao (Stress test với 8 Peer, chạy 3 phút):

```
powershell -ExecutionPolicy Bypass -File .\start-churn.ps1 -Peers  
8 -Duration 180
```

PHỤ LỤC B. HƯỚNG DẪN SỬ DỤNG GIAO DIỆN

B.1. Khi mới khởi động:

- Peer tự động kết nối đến Bootstrap Server (nếu có Internet) và quét UDP broadcast trong LAN.
- Danh sách peer online hiển thị ở Sidebar trái.

B.2. Chat 1-1:

- Nhấp đúp vào tên peer trong danh sách.
- Gõ tin nhắn vào ô dưới cùng, nhấn Enter hoặc nút Gửi.
- Tin nhắn được mã hóa E2EE, hiển thị dấu tích xanh khi đã đến nơi.
- Nhấn vào biểu tượng ổ khóa bên cạnh tin nhắn để xem chi tiết mã hóa (ciphertext).

B.3. Tạo phòng chat nhóm:

- Nút “+ New Room” → nhập tên phòng → hệ thống sinh room key (mã số).
- Sao chép room key để gửi cho bạn bè (qua kênh ngoài).

B.4. Tham gia phòng nhóm:

- Nút “Join Room” → dán room key → OK.
- Các tin nhắn trong phòng chỉ hiển thị cho thành viên có key.

B.5. Gửi file:

- Biểu tượng đính kèm (+) → chọn file (ảnh, tài liệu <5MB).
- File được chuyển thành Base64, mã hóa AES-256-GCM và gửi qua P2P.

- Bên nhận xem trực tiếp ảnh hoặc tải về.

B.6. Broadcast toàn mạng:

- Chọn kênh “NETWORK_BROADCAST” từ sidebar.
- Tin nhắn gửi đi sẽ đến mọi peer đang online (có kiểm soát TTL=3, chống lặp).

B.7. Thoát ứng dụng:

- Đóng cửa sổ → hệ thống tự gửi LEAVE đến các peer và Bootstrap Server, dọn dẹp tài nguyên.
- Chọn nút “Disconnect” → Hệ thống hiển thị thông báo xác nhận thoát ứng dụng